

Unit 7: Tuning and Moving

SQL Tuning

- **SQL tuning** is the iterative process of improving SQL statement performance to meet specific, measurable, and achievable goals.
- SQL tuning implies fixing problems in deployed applications. In contrast, application design sets the security and performance goals before deploying an application.
- SQL tuning is improving SQL statement performance to meet specific, measurable, and achievable goals.
- SQL tuning is the attempt to diagnose and repair SQL statements that fail to meet a performance standard.

Purpose of SQL Tuning

- A SQL statement becomes a problem when it fails to perform according to a predetermined and measurable standard.
- After you have identified the problem, a typical tuning session has one of the following goals:
 - ☑ **Reduce user response time**, which means decreasing the time between when a user issues a statement and receives a response
 - ☑ **Improve throughput**, which means using the least amount of resources necessary to process all rows accessed by a statement
- For a response time problem, consider an online book seller application that hangs for three minutes after a customer updates the shopping cart. Contrast with a three-minute parallel query in a data warehouse that consumes all of the database host CPU, preventing other queries from running. In each case, the user response time is three minutes, but the cause of the problem is different, and so is the tuning goal.

SQL Tuning Tasks

- The specifics of a tuning session depend on many factors, including whether you tune proactively or reactively.

- In **proactive SQL tuning**, you regularly use SQL Tuning Advisor to determine whether you can make SQL statements perform better. In **reactive SQL tuning**, you correct a SQL-related problem that a user has experienced.
- Whether you tune proactively or reactively, a typical SQL tuning session involves all or most of the following tasks:

1. **Identifying high-load SQL statements**

Review past execution history to find the statements responsible for a large share of the application workload and system resources.

2. **Gathering performance-related data**

The optimizer statistics are crucial to SQL tuning. If these statistics do not exist or are no longer accurate, then the optimizer cannot generate the best plan. Other data relevant to SQL performance include the structure of tables and views that the statement accessed, and definitions of any indexes available to the statement.

3. **Determining the causes of the problem**

Typically, causes of SQL performance problems include:

- Inefficiently designed SQL statements

If a SQL statement is written so that it performs unnecessary work, then the optimizer cannot do much to improve its performance. Examples of inefficient design include

 - Neglecting to add a join condition, which leads to a Cartesian join
 - Using hints to specify a large table as the driving table in a join
 - Specifying UNION instead of UNION ALL
 - Making a subquery execute for every row in an outer query
- *Suboptimal execution plans*

The query optimizer (also called the optimizer) is internal software that determines which execution plan is most efficient. Sometimes the optimizer chooses a plan with a suboptimal access path, which is the means by which the database retrieves data from the database. For example, the plan for a query predicate with low selectivity may use a full table scan on a large table instead of an index.

You can compare the execution plan of an optimally performing SQL statement to the plan of the statement when it performs suboptimally. This comparison,

along with information such as changes in data volumes, can help identify causes of performance degradation.

- *Missing SQL access structures*

Absence of SQL access structures, such as indexes and materialized views, is a typical reason for suboptimal SQL performance. The optimal set of access structures can improve SQL performance by orders of magnitude.

- *Stale optimizer statistics*

Statistics gathered by DBMS_STATS can become stale when the statistics maintenance operations, either automatic or manual, cannot keep up with the changes to the table data caused by DML. Because stale statistics on a table do not accurately reflect the table data, the optimizer can make decisions based on faulty information and generate suboptimal execution plans.

- *Hardware problems*

Suboptimal performance might be connected with memory, I/O, and CPU problems.

4. *Defining the scope of the problem*

The scope of the solution must match the scope of the problem. Consider a problem at the database level and a problem at the statement level. For example, the shared pool is too small, which causes cursors to age out quickly, which in turn causes many hard parses. Using an initialization parameter to increase the shared pool size fixes the problem at the database level and improves performance for all sessions. However, if a single SQL statement is not using a helpful index, then changing the optimizer initialization parameters for the entire database could harm overall performance. If a single SQL statement has a problem, then an appropriately scoped solution addresses just this problem with this statement.

5. *Implementing corrective actions for suboptimally performing SQL statements*

These actions vary depending on circumstances. For example, you might rewrite a SQL statement to be more efficient, avoiding unnecessary hard parsing by rewriting the statement to use bind variables. You might also use equijoins, remove functions from WHERE clauses, and break a complex SQL statement into multiple simple statements.

In some cases, you improve SQL performance not by rewriting the statement, but by restructuring schema objects. For example, you might index a new [access path](#), or reorder columns in a concatenated index. You might also partition a table, introduce derived values, or even change the database design.

6. *Preventing SQL performance regressions*

To ensure optimal SQL performance, verify that execution plans continue to provide optimal performance, and choose better plans if they come available. You can achieve these goals using optimizer statistics, SQL profiles, and SQL plan baselines.

SQL Tuning Tools

- SQL tuning tools are either **automated or manual**.
- In this context, a tool is automated if the database itself can provide diagnosis, advice, or corrective actions. A manual tool requires you to perform all of these operations.
- All tuning tools depend on the basic tools of the dynamic performance views, statistics, and metrics that the database instance collects. The database itself contains the data and metadata required to tune SQL statements.

Automated SQL Tuning Tools

- Oracle Database provides several advisors relevant for SQL tuning.
- Additionally, SQL plan management is a mechanism that can prevent performance regressions and also help you to improve SQL performance.
- All of the automated SQL tuning tools can use SQL tuning sets as input. A SQL tuning set (STS) is a database object that includes one or more SQL statements along with their execution statistics and execution context.

Following are important Automated SQL Tuning Tools:

- *Automatic Database Diagnostic Monitor (ADDM)*
- *SQL Tuning Advisor*
- *SQL Access Advisor*
- *Automatic Indexing*
- *SQL Plan Management*
- *SQL Performance Analyzer*

Automatic Database Diagnostic Monitor (ADDM)

- ADDM is self-diagnostic software built into Oracle Database.
- ADDM can automatically locate the root causes of performance problems, provide recommendations for correction, and quantify the expected benefits. ADDM also identifies areas where no action is necessary.
- ADDM and other advisors use **Automatic Workload Repository (AWR)**, which is an infrastructure that provides services to database components to collect, maintain, and use statistics. ADDM examines and analyzes statistics in AWR to determine possible performance problems, including high-load SQL.
- For example, you can configure ADDM to run nightly. In the morning, you can examine the latest ADDM report to see what might have caused a problem and if there is a recommended fix. The report might show that a particular SELECT statement consumed a huge amount of CPU, and recommend that you run SQL Tuning Advisor.

SQL Tuning Advisor

- SQL Tuning Advisor is internal diagnostic software that identifies problematic SQL statements and recommends how to improve statement performance.
- When run during database maintenance windows as an automated maintenance task, SQL Tuning Advisor is known as **Automatic SQL Tuning Advisor**.
- SQL Tuning Advisor takes one or more SQL statements as an input and invokes the **Automatic Tuning Optimizer** to perform SQL tuning on the statements. The advisor performs the following types of analysis:
 - Checks for missing or stale statistics
 - Builds SQL profiles
 - A **SQL profile** is a set of auxiliary information specific to a SQL statement. A SQL profile contains corrections for suboptimal optimizer estimates discovered during Automatic SQL Tuning. This information can improve optimizer estimates for cardinality, which is the number of rows that is estimated to be or actually is returned by an operation in an execution plan, and selectivity. These improved estimates lead the optimizer to select better plans.

- Explores whether a different access path can significantly improve performance
- Identifies SQL statements that lend themselves to suboptimal plans
- The output is in the form of advice or recommendations, along with a rationale for each recommendation and its expected benefit. The recommendation relates to a collection of statistics on objects, creation of new indexes, restructuring of the SQL statement, or creation of a SQL profile. You can choose to accept the recommendations to complete the tuning of the SQL statements.

SQL Access Advisor

- SQL Access Advisor is internal diagnostic software that recommends which materialized views, indexes, and materialized view logs to create, drop, or retain.
- SQL Access Advisor takes an actual workload as input, or the advisor can derive a hypothetical workload from the schema. SQL Access Advisor considers the trade-offs between space usage and query performance, and recommends the most cost-effective configuration of new and existing materialized views and indexes. The advisor also makes recommendations about partitioning.

Automatic Indexing

- Oracle Database can constantly monitor the application workload, creating and managing indexes automatically.
- Creating indexes manually requires deep knowledge of the data model, application, and data distribution. Often DBAs make choices about which indexes to create, and then never revise their choices. As a result, opportunities for improvement are lost, and unnecessary indexes can become a performance liability. Automatic index management solves this problem.

SQL Plan Management

- SQL plan management is a preventative mechanism that enables the optimizer to automatically manage execution plans, ensuring that the database uses only known or verified plans.

- This mechanism can build a **SQL plan baseline**, which contains one or more accepted plans for each SQL statement. By using baselines, SQL plan management can prevent plan regressions from environmental changes, while permitting the optimizer to discover and use better plans.

SQL Performance Analyzer

- SQL Performance Analyzer determines the effect of a change on a SQL workload by identifying performance divergence for each SQL statement.
- System changes such as upgrading a database or adding an index may cause changes to execution plans, affecting SQL performance. By using SQL Performance Analyzer, you can accurately forecast the effect of system changes on SQL performance. Using this information, you can tune the database when SQL performance regresses, or validate and measure the gain when SQL performance improves.

Manual SQL Tuning Tools

In some situations, you may want to run manual tools in addition to the automated tools. Alternatively, you may not have access to the automated tools.

Followings are the Manual SQL Tuning Tools

- ***Execution Plans***
- ***Real-Time SQL Monitoring and Real-Time Database Operations***
- ***Application Tracing***
- ***Optimizer Hints***

Execution Plans

- Execution plans are the principal diagnostic tool in manual SQL tuning. For example, you can view plans to determine whether the optimizer selects the plan you expect, or identify the effect of creating an index on a table.
- You can display execution plans in multiple ways. The following tools are the most commonly used:
 - **DBMS_XPLAN**

You can use the DBMS_XPLAN package methods to display the execution plan generated by the EXPLAIN PLAN command and query of V\$SQL_PLAN.

- **EXPLAIN PLAN**

This SQL statement enables you to view the execution plan that the optimizer would use to execute a SQL statement without actually executing the statement.

- **V\$SQL_PLAN and related views**

These views contain information about executed SQL statements, and their execution plans, that are still in the shared pool.

- **AUTOTRACE**

The AUTOTRACE command in SQL*Plus generates the execution plan and statistics about the performance of a query. This command provides statistics such as disk reads and memory reads.

Real-Time SQL Monitoring and Real-Time Database Operations

- The Real-Time SQL Monitoring feature of Oracle Database enables you to monitor the performance of SQL statements while they are executing. By default, SQL monitoring starts automatically when a statement runs in parallel, or when it has consumed at least 5 seconds of CPU or I/O time in a single execution.
- A **database operation** is a set of database tasks defined by end users or application code, for example, a batch job or Extraction, Transformation, and Loading (ETL) processing. You can define, monitor, and report on database operations. Real-Time Database Operations provides the ability to monitor composite operations automatically. The database automatically monitors parallel queries, DML, and DDL statements as soon as execution begins.
- Oracle Enterprise Manager Cloud Control (Cloud Control) provides easy-to-use SQL monitoring pages. Alternatively, you can monitor SQL-related statistics using the V\$SQL_MONITOR and V\$SQL_PLAN_MONITOR views. You can use these views with the following views to get more information about executions that you are monitoring:
 - V\$ACTIVE_SESSION_HISTORY
 - V\$SESSION
 - V\$SESSION_LONGOPS

- V\$SQL
- V\$SQL_PLAN

Application Tracing

- A **SQL trace** file provides performance information on individual SQL statements: parse counts, physical and logical reads, misses on the library cache, and so on.
- Trace files are sometimes useful for diagnosing SQL performance problems. You can enable and disable SQL tracing for a specific session using the DBMS_MONITOR or DBMS_SESSION packages. Oracle Database implements tracing by generating a trace file for each server process when you enable the tracing mechanism.
- Oracle Database provides the following command-line tools for analyzing trace files:

TKPROF

This utility accepts as input a trace file produced by the SQL Trace facility, and then produces a formatted output file.

trcsess

This utility consolidates trace output from multiple trace files based on criteria such as session ID, client ID, and service ID. After trcsess merges the trace information into a single output file, you can format the output file with TKPROF. trcsess is useful for consolidating the tracing of a particular session for performance or debugging purposes.

- End-to-End Application Tracing simplifies the process of diagnosing performance problems in multitier environments. In these environments, the middle tier routes a request from an end client to different database sessions, making it difficult to track a client across database sessions. End-to-End application tracing uses a client ID to uniquely trace a specific end-client through all tiers to the database.

Optimizer Hints

- A **hint** is an instruction passed to the optimizer through comments in a SQL statement.

- Hints enable you to make decisions normally made automatically by the optimizer. In a test or development environment, hints are useful for testing the performance of a specific access path. For example, you may know that a specific index is more selective for certain queries. In this case, you may use hints to instruct the optimizer to use a better execution plan, as in the following example:

```
SELECT /*+ INDEX (employees emp_department_ix) */
       employee_id, department_id
FROM   employees
WHERE  department_id > 50;
```

- Sometimes the database may not use a hint because of typos, invalid arguments, conflicting hints, and hints that are made invalid by transformations. Starting in Oracle Database 19c, you can generate a report about which hints were used or not used during plan generation.

User Interfaces to SQL Tuning Tools

- Cloud Control is a system management tool that provides centralized management of a database environment. Cloud Control provides access to most tuning tools.
- By combining a graphical console, Oracle Management Servers, Oracle Intelligent Agents, common services, and administrative tools, Cloud Control provides a comprehensive system management platform.
- You can access all SQL tuning tools using a command-line interface. For example, the DBMS_SQLTUNE package is the command-line interface for SQL Tuning Advisor.
- Oracle recommends Cloud Control as the best interface for database administration and tuning. In cases where the command-line interface better illustrates a particular concept or task, this manual uses command-line examples. However, in these cases the tuning tasks include a reference to the principal Cloud Control page associated with the task.

Cancel a SQL Statement in a Session

- Before Oracle 18c, terminating a specific SQL query within a session without killing the entire session could be a challenge. The typical method involved finding the session's SID and SERIAL#, then killing the session using the ALTER SYSTEM KILL SESSION command. However, this approach terminates the entire session, which might not always be desirable.
- After 18c, instead of kill the originating session we can cancel the particular SQL of another session using “ALTER SYSTEM CANCEL SQL ‘SID, SERIAL’”. If we are cancel a DML statement, the statement has been rolled backed.
- Starting with Oracle 18c, Oracle introduced the ability to cancel specific SQL statements in a session without terminating the entire session. This can be especially useful for managing resource-intensive or incorrect queries without affecting other operations the session might be handling.
- The basic syntax of the **ALTER SYSTEM CANCEL SQL** statement is shown below.
ALTER SYSTEM CANCEL SQL 'SID, SERIAL[, @INST_ID][, SQL_ID]';

This statement allows you to cancel a specific SQL statement currently executing within an Oracle database session. Here's a breakdown of the components:

- **ALTER SYSTEM:** Keyword indicating you're modifying system settings.
- **CANCEL SQL:** Specifies the action you want to perform, which is canceling a running SQL statement.
- **'SID, SERIAL':** Mandatory arguments that identify the specific SQL statement to cancel.
 - **SID:** The session identifier of the session running the SQL statement.
 - **SERIAL:** The serial number of the SQL statement within the session.
You can find these values using tools like `SELECT sid, serial# FROM v$session WHERE username = '<username>'`.
- **[, @INST_ID]:** Optional clause.
 - **@INST_ID:** If your Oracle instance has a non-default instance ID, specify it here to target the specific instance. Otherwise, it defaults to the current instance.
- **[, SQL_ID]:** Optional clause.

- **SQL_ID:** If you know the SQL_ID of the specific statement you want to cancel, you can use it here instead of relying on SID and SERIAL.

You can find the SQL_ID using tools like `SELECT sql_id FROM v$sql WHERE sid = <sid> AND serial# = <serial>`.

- By running this statement, you can terminate a specific SQL statement that might be causing performance issues or is no longer needed. However, it's important to note that this action won't terminate the entire session. The session will continue running after the targeted SQL statement is canceled.

Steps to Cancel a Specific SQL Statement

1. *Identify the Problematic Session and SQL Statement:*

- First, you need to identify the session and the SQL statement you want to cancel. You can do this by querying the V\$SESSION and V\$SQL views.

```
SELECT s.sid, s.serial#, s.username, s.status, s.osuser,
       s.machine, s.program, q.sql_id, q.sql_text
FROM v$session s
JOIN v$sql q ON s.sql_id = q.sql_id
WHERE s.status = 'ACTIVE';
```

This query returns the session details along with the currently running SQL statement.

2. *Cancel the Specific SQL Statement:*

- Once you have identified the SID and SERIAL# of the session, as well as the SQL_ID of the query you want to cancel, you can use the ALTER SYSTEM CANCEL SQL command to cancel that specific statement.

`ALTER SYSTEM CANCEL SQL 'sid,serial#,sql_id';`

For example, if the SID is 123, the SERIAL# is 4567, and the SQL_ID is 'abcd1234', the command would be:

`ALTER SYSTEM CANCEL SQL '123,4567,abcd1234';`

Important Considerations

- ☞ **Rollback of DML Statements:** When you cancel a Data Manipulation Language (DML) statement (such as INSERT, UPDATE, or DELETE), the statement is rolled back, ensuring data consistency.
- ☞ **Privileges:** You need the necessary system privileges (ALTER SYSTEM) to cancel SQL statements in other sessions. Typically, this is a DBA-level privilege.
- ☞ **Impact:** Canceling a query might have implications depending on what the query was doing. For instance, canceling a long-running SELECT might be harmless, but canceling a DML operation will result in a rollback of any changes made by that operation.

The Oracle Optimizer

- ☞ *SQL processing* is the parsing, optimization, row source generation, and execution of a SQL statement.
- ☞ The *query optimizer* (called simply the optimizer) is built-in database software that determines the most efficient method for a SQL statement to access requested data.

Purpose of the Query Optimizer

- ☞ The optimizer attempts to generate the most optimal execution plan for a SQL statement.
- ☞ The optimizer choose the plan with the lowest cost among all considered candidate plans. The optimizer uses available statistics to calculate cost. For a specific query in a given environment, the cost computation accounts for factors of query execution such as I/O, CPU, and communication.
- ☞ For example, a query might request information about employees who are managers. If the optimizer statistics indicate that 80% of employees are managers, then the optimizer may decide that a full table scan is most efficient. However, if statistics indicate that very few employees are managers, then reading an index followed by a table access by rowid may be more efficient than a full table scan.
- ☞ Because the database has many internal statistics and tools at its disposal, the optimizer is usually in a better position than the user to determine the optimal

method of statement execution. For this reason, all SQL statements use the optimizer.

Cost-Based Optimization

- ☞ **Query optimization** is the process of choosing the most efficient means of executing a SQL statement.
- ☞ SQL is a nonprocedural language, so the optimizer is free to merge, reorganize, and process in any order. The database optimizes each SQL statement based on statistics collected about the accessed data. The optimizer determines the optimal plan for a SQL statement by examining multiple access methods, such as full table scan or index scans, different join methods such as nested loops and hash joins, different join orders, and possible transformations.
- ☞ For a given query and environment, the optimizer assigns a relative numerical cost to each step of a possible plan, and then factors these values together to generate an overall cost estimate for the plan. After calculating the costs of alternative plans, the optimizer chooses the plan with the lowest cost estimate. For this reason, the optimizer is sometimes called the *cost-based optimizer (CBO)* to contrast it with the legacy *rule-based optimizer (RBO)*.

Note: The optimizer may not make the same decisions from one version of Oracle Database to the next. In recent versions, the optimizer might make different decision because better information is available and more optimizer transformations are possible.

SQL Plan Directives

- ☞ A SQL plan directive is additional information and instructions that the optimizer can use to generate a more optimal plan.
- ☞ The directive is a “note to self” by the optimizer that it is misestimating cardinalities of certain types of predicates, and also a reminder to DBMS_STATS to gather statistics needed to correct the misestimates in the future. For example, when joining two tables that have a data skew in their join columns, a SQL plan directive can direct the optimizer to use dynamic statistics to obtain a more accurate join cardinality estimate.

- ☞ SQL Plan Directives (SPDs) in Oracle are a feature designed to improve the optimizer's ability to generate efficient execution plans, particularly for complex queries or those involving skewed data distributions. Introduced in Oracle 12c and enhanced in subsequent versions, including Oracle 19c, SPDs automatically gather and use additional information about the data and query execution environment to make more informed decisions.

How the Database Uses SQL Plan Directives

- ☞ When compiling a SQL statement, if the optimizer sees a directive, then it obeys the directive by gathering additional information.
- ☞ The optimizer uses directives in the following ways:

Dynamic statistics

- The optimizer uses dynamic statistics whenever it does not have sufficient statistics corresponding to the directive. For example, the cardinality estimates for a query whose predicate contains a specific pair of columns may be significantly wrong. A SQL plan directive indicates that the whenever a query that contains these columns is parsed, the optimizer needs to use dynamic sampling to avoid a serious cardinality misestimate.
- Dynamic statistics have some performance overhead. Every time the optimizer hard parses a query to which a dynamic statistics directive applies, the database must perform the extra sampling.
- Starting in Oracle Database 12c Release 2 (12.2), the database writes statistics from *adaptive dynamic sampling* to the SQL plan directives store, making them available to other queries.

Column groups

- The optimizer examines the query corresponding to the directive. If there is a missing column group, and if the DBMS_STATS preference AUTO_STAT_EXTENSIONS is set to ON (the default is OFF) for the affected table, then the optimizer automatically creates this column group the next

time DBMS_STATS gathers statistics on the table. Otherwise, the optimizer does not automatically create the column group.

- If a column group exists, then the next time this statement executes, the optimizer uses the *column group statistics* in place of the SQL plan directive when possible (equality predicates, GROUP BY, and so on). In subsequent executions, the optimizer may create additional SQL plan directives to address other problems in the plan, such as join or GROUP BY cardinality misestimates.

- ☞ When the problem that occasioned a directive is solved, either because a better directive exists or because a histogram or extension exists, the DBA_SQL_PLAN_DIRECTIVES.STATE value changes from USABLE to SUPERSEDED. More information about the directive state is exposed in the DBA_SQL_PLAN_DIRECTIVES.NOTES column.

SQL Plan Directive Maintenance

- ☞ The database automatically creates SQL plan directives. You cannot create them manually.
- ☞ The database initially creates directives in the shared pool. The database periodically writes the directives to the SYSAUX tablespace. The database automatically purges any SQL plan directive that is not used after the specified number of weeks (SPD_RETENTION_WEEKS), which is 53 by default.
- ☞ You can manage directives by using the DBMS_SPD package. For example, you can:
 - Enable and disable SQL plan directives (ALTER_SQL_PLAN_DIRECTIVE)
 - Change the retention period for SQL plan directives (SET_PREFS)
 - Export a directive to a staging table (PACK_STGTAB_DIRECTIVE)
 - Drop a directive (DROP_SQL_PLAN_DIRECTIVE)
 - Force the database to write directives to disk (FLUSH_SQL_PLAN_DIRECTIVE)

Adaptive Execution Plans

- An **execution plan** is the sequence of operations that the database performs to run a SQL statement.
- The EXPLAIN PLAN statement displays execution plans that the optimizer chooses for SELECT, UPDATE, INSERT, and DELETE statements.
 - ☞ EXPLAIN PLAN output shows how the database would have run the SQL statement when the statement was explained. Because of differences in the execution environment and explain plan environment, the explained plan can differ from the actual plan used during statement execution.
 - ☞ When the EXPLAIN PLAN statement is issued, the optimizer chooses an execution plan and then inserts a row describing each step of the execution plan into a specified plan table. You can also issue the EXPLAIN PLAN statement as part of the SQL trace facility.
 - ☞ The EXPLAIN PLAN statement is a DML statement rather than a DDL statement. Therefore, Oracle Database does not implicitly commit the changes made by an EXPLAIN PLAN statement.
- An **adaptive query plan** contains multiple predetermined subplans, and an optimizer statistics collector. Based on the statistics collected during execution, the dynamic plan coordinator chooses the best plan at run time.
 - ☞ An **adaptive query plan** enables the optimizer to make a plan decision for a statement during execution.
 - ☞ **Adaptive query plans** enable the optimizer to fix some classes of problems at run time. Adaptive plans are enabled by default.
- The **adaptive optimizer** is a feature of the optimizer that enables it to adapt plans based on run-time statistics. All adaptive mechanisms can execute a final plan for a statement that differs from the default plan.
- An **adaptive query plan** chooses among subplans during the current statement execution. In contrast, *automatic reoptimization* changes a plan only on executions that occur after the current statement execution.

Notes

- ☞ **Adaptive optimizer** is a feature of the optimizer that enables it to adapt plans based on run-time statistics.
- ☞ **Adaptive query plan** is a set of capabilities that enables the *adaptive optimizer* to make run-time adjustments to execution plans and discover additional information that can lead to better statistics. Adaptive optimization is helpful when existing statistics are not sufficient to generate an optimal plan.
- ☞ **Automatic reoptimization** is the ability of the optimizer to automatically change a plan on subsequent executions of a SQL statement. Automatic reoptimization can fix any suboptimal plan chosen due to incorrect optimizer estimates, from a suboptimal distribution method to an incorrect choice of degree of parallelism
- You can determine whether the database used adaptive query optimization for a SQL statement based on the comments in the *Notes* section of plan. The comments indicate whether row sources are dynamic, or whether automatic reoptimization adapted a plan.

Displaying Adaptive Query Plans

Follow the tutorial given in [this link](#) for detail

<https://docs.oracle.com/en/database/oracle/oracle-database/19/tgsql/generating-and-displaying-execution-plans.html#GUID-83F88700-3902-4D19-8182-AF2B92AEA7EB>

Tuning SQL Statements Using SQL Tuning Advisor

- You can use SQL Tuning Advisor to tune one or more SQL statements. When tuning multiple statements, SQL Tuning Advisor does not recognize interdependencies between the SQL statements. Instead, SQL Tuning Advisor provides a convenient way to obtain tuning advice for a large number of SQL statements.
- Oracle Database can generate SQL tuning reports automatically. Automatic SQL Tuning runs during system **maintenance windows** as an automated maintenance task, searching for ways to improve the execution plans of

high-load SQL statements. A maintenance window is a contiguous time interval during which automated maintenance tasks are run.

Tuning SQL Manually Using SQL Tuning Advisor

Automatic Database Diagnostic Monitor (ADDM) automatically identifies high-load SQL statements. If ADDM identifies such statements, then click **Schedule/Run SQL Tuning Advisor** on the Recommendation Detail page to run SQL Tuning Advisor.

To tune SQL statements manually using SQL Tuning Advisor:

1. Access the Database Home page.
See "Accessing the Database Home Page" for more information.
2. From the **Performance** menu, select **Advisors Home**
If the Database Login page appears, then log in as a user with administrator privileges. The Advisor Central page appears.
3. In the Advisors section, click **SQL Advisors**. The SQL Advisors page appears.
4. In the SQL Tuning Advisor section, click **SQL Tuning Advisor**.
The Schedule SQL Tuning Advisor page appears.

Schedule SQL Tuning Advisor Cancel Submit

Specify the following parameters to schedule a job to run the SQL Tuning Advisor.

* Name	SQL_TUNING_1235181904085
Description	
* SQL Tuning Set	
SQL Tuning Set Description	
SQL Statements	0
Counts	

Overview

The SQL Tuning Advisor analyzes individual SQL statements, and suggests indexes, SQL profiles, restructured SQL, and statistics that improve the performance of the SQL statements.

The SQL Tuning Advisor operates on a collection of SQL. You can choose a SQL Tuning Set to run the advisor. If you do not have a SQL Tuning Set with the desired SQL for running the advisor, you can create a new one.

You can click on one of the following sources, which will lead you to a data source where you can tune SQL statements using the SQL Tuning Advisor.

[Top Activity](#) [Historical SQL \(AWR\)](#) [SQL Tuning Sets](#)

5. In the **Name** field, enter a name for the SQL tuning task.
If unspecified, then SQL Tuning Advisor uses a system-generated name.
6. Do one of the following:
 - To run a SQL tuning task for one or more high-load SQL statements, under Overview click **Top Activity**.
The Top Activity page appears.
Under Top SQL, select the SQL statement that you want to tune.
In the Actions list, select **Schedule SQL Tuning Advisor** and click **Go**.

- To run a SQL tuning task for historical SQL statements from the Automatic Workload Repository (AWR), under Overview click **Historical SQL (AWR)**.

The Historical SQL (AWR) page appears.

Under Historical SQL (AWR), click the band below the chart, and select the 24-hour interval for which you want to view SQL statements that ran on the database. Under Detail for Selected 24 Hour Interval, select the SQL statement you want to tune, and click **Schedule SQL Tuning Advisor**.

- To run a SQL tuning task for a SQL tuning set, under Overview click **SQL Tuning Sets**.

The SQL Tuning Sets page appears.

Select the SQL tuning set that contains the SQL statements you want to tune and then click **Schedule SQL Tuning Advisor**.

The Schedule SQL Tuning Advisor page reappears.

- To display the SQL text of the selected statement, expand **SQL Statements**.

SQL Statements	
SQL Text	Parsing Schema
select * from sales where quantity_sold < 5 union select * from sales where quantity_sold > 500	SH

- Under Scope, select the scope of tuning to perform. Do one of the following:

- Select **Limited**.

A limited scope takes approximately 1 second to tune each SQL statement but does not recommend a SQL profile.

- Select **Comprehensive**, and then set a time limit (in minutes) for each SQL statement in the **Time Limit per Statement** field, and a total time limit (in minutes) in the **Total Time Limit** field. Note that setting the time limit too small may affect the quality of the recommendations.

Comprehensive mode may take several minutes to tune a single SQL statement. This mode is both time and resource intensive because each query must be hard-parsed. You should only use comprehensive scope for high-load SQL statements that have a significant impact on the entire system.

- Under Schedule, do one of the following:

- Select **Immediately** and then click **Submit** to run the SQL tuning task immediately.

The Processing: SQL Tuning Advisor Task page appears.

- Select **Later** to schedule a specific time in the future, and then click **OK**.

- From the Performance menu, select **Advisors Home**.

The Advisor Central page appears.

Under Advisor Tasks, the Results sections lists the result of advisors.

Results								
View Result		Delete	Actions	Re-schedule	Go	Previous 1-25 of 1346 Next 25		
Select	Name	Advisory Type	Description	User	Status	Start Time	Duration (seconds)	Expires In (days)
☒	SQL_TUNING_1329403878841	SQL Tuning Advisor		DBA1	COMPLETED	Feb 16, 2012 8:55:43 AM	17	30

11. Select a SQL Tuning Advisor type result from the table and then click **View Result**.

The Recommendations for SQL ID page appears.

Select Recommendation								
Original Explain Plan (Annotated)								
Select	Type	Findings	Recommendations	Rationale	Benefit (%)	Other Statistics	New Explain Plan	Compare Explain Plans
☒	Restructure SQL	An expensive "UNION" operation was found at line ID 1 of the execution plan.	Consider using "UNION ALL" instead of "UNION", if duplicates are allowed or uniqueness is guaranteed.	"UNION" is an expensive and blocking operation because it requires elimination of duplicate rows. "UNION ALL" is a cheaper alternative, assuming that duplicates are allowed or uniqueness is guaranteed.				

If you used a SQL tuning set, then multiple recommendations may be shown. To help you decide whether to implement a recommendation, an estimated benefit of implementing the recommendation is displayed in the Benefit (%) column. The Rationale column displays an explanation of why the recommendation is made.

12. To **implement the recommendation**, do one of the following:

- If an automated solution is recommended, then click **Implement**. A confirmation page appears. Click **Yes** to confirm the change.
- If a manual solution is recommended, then consider implementing the recommendation.

Automatic SQL Tuning Results

- By analyzing data in the Automatic Workload Repository (AWR), the database can identify routine maintenance tasks. The automated maintenance tasks infrastructure (known as AutoTask) schedules these tasks to run in maintenance windows.
- Maintenance windows are Oracle Scheduler time intervals that belong to the window group named MAINTENANCE_WINDOW_GROUP. By default, one window is scheduled for each day of the week. You can customize attributes of these maintenance windows, including start and end times, frequency, and days of the week.
- By default, AutoTask runs the following automated maintenance tasks in all maintenance windows:

- Optimizer Statistics Collection
- Segment Advisor
- SQL Tuning Advisor
- You can view the results of automated execution of SQL Tuning Advisor on observed high-load SQL statements.

To view automatic SQL tuning results:

1. Access the Database Home page.

See "Accessing the Database Home Page" for more information.

2. From the **Performance** page, select **Advisors Home**.

If the Database Login page appears, then log in as a user with administrator privileges. The Advisor Central page appears.

3. Under Advisors, click **SQL Advisors**.

The SQL Advisors page appears.

4. Under SQL Tuning Advisor, click **Automatic SQL Tuning Results**.

The Automatic SQL Tuning Result Summary page appears.

The top half of the page includes sections for the status and activity summary of the SQL Tuning task.

Automatic SQL Tuning Result Summary

The Automatic SQL Tuning runs during system maintenance windows as an automated maintenance task, searching for ways to improve the execution plans of high-load SQL statements.

Task Status

Automatic SQL Tuning (SYS_AUTO_SQL_TUNING_TASK) is currently **Enabled** [Configure](#)

Automatic Implementation of SQL Profiles is currently **Disabled** [Configure](#)

Key SQL Profiles [1](#) [Implement All](#)

Summary Time Period

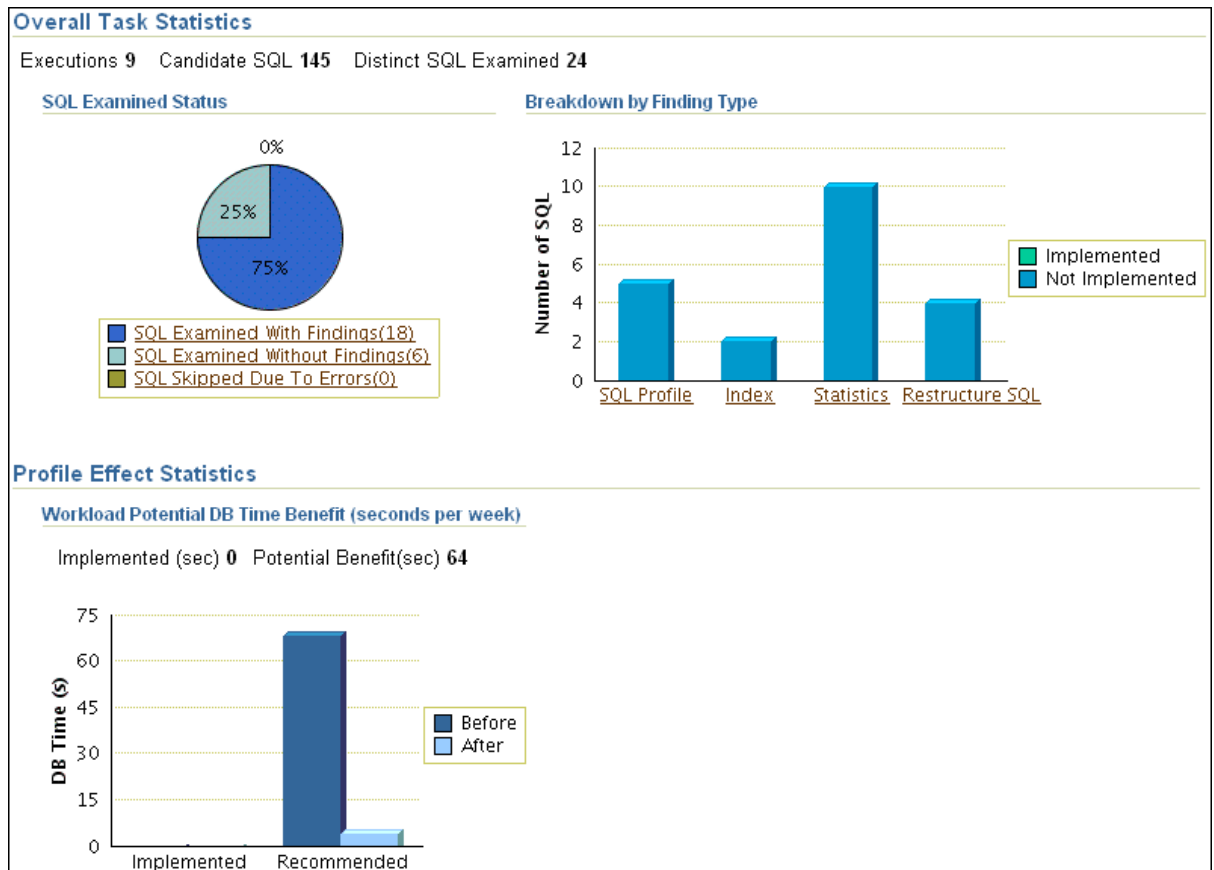
Choose a time period to focus the graphs and statistics below on a specific range of tuning results. Drill down to view focused results or see the results for all SQLs by clicking the "View Report" button.

Time Period [Go](#) [View Report](#)

Begin Date **Feb 19, 2012 7:00:02 PM (UTC-08:00)** End Date **Feb 23, 2012 10:29:34 AM (UTC-08:00)**

5. In the Time Period list, select **All** and then click **Go**.

The Overall Task Statistics and Profile Effect Statistics sections are refreshed.



- Optionally, in the Task Status section, click **Configure** to change the attributes of the Automatic SQL Tuning task.

The Automated Maintenance Tasks Configuration page appears.

In this page, you can enable or disable the Automatic SQL Tuning task and specify which days it should run. Click **Apply** or **Revert** to return to the previous page.

- In the Task Activity Summary section, leave **All** selected for the **Time Period** and then click **View Report**.

The Automatic SQL Tuning Result Details page appears.

The page lists SQL statements that have been automatically selected by the database as candidates for SQL tuning.

Automatic SQL Tuning Result Details: All Analyzed SQLs													
Begin Date Feb 19, 2012 7:00:02 PM (UTC-08:00)					End Date Feb 23, 2012 12:16:57 PM (UTC-08:00)								
Recommendations													
Only profiles that significantly improve SQL performance were implemented.													
View Recommendations Implement All SQL Profiles													
Select	SQL Text	Parsing Schema	SQL ID	Weekly DB Time Benefit(sec)	Per-Execution % Benefit	Statistics	SQL Profile	Index	Restructure SQL	Miscellaneous	Timed Out	Error	Date
☒	select sum(quantity_sold) from sales s, ...	SH	fudq5z56g642p	60.38	96		(96%) ✓	(62%) ✓					2/20/2012 10:00:02 AM
☐	select * from sales where amount_sold = ...	SH	bzmnj0nbvmz8t	3.97	60			(60%) ✓					2/20/2012 10:00:02 AM
☐	/* OracleOEM */ SELECT /*+ INDEX(ts) */...	DBSNMP	ab7ktdksu27i	2.36	87	✓	(87%) ✓			✓			2/20/2012 7:00:00 PM
☐	SELECT TASK_LIST.TASK_ID FROM (SELECT /*+ ...	DBSNMP	bqtc5q2ias08u	0.95	98	✓	(98%) ✓			✓			2/23/2012 10:00:02 AM
☐	SELECT NVL(SUM(e.non_exempt_violations_...	SYSMAN	b4m9s8bfr8x7	0.40	67	✓	(67%) ✓						2/19/2012 7:00:02 PM
☐	SELECT :B1 TASK_ID, F.FINDING_ID FINDING...	DBSNMP	a8i39qb13tqr	0.23	24		(24%) ✓		✓	✓			2/20/2012 2:00:02 PM

8. Under Recommendations, select a SQL statement and then click **View Recommendations**.

The Recommendations for SQL ID page appears

Recommendations for SQL ID:fudq5z56g642p							
							Return
Only one recommendation should be implemented.							
SQL Text							
select sum(quantity_sold) from sales s, products p where s.prod_id = p.prod_id and s.amount_sold > 20000 and p.prod_name = 'Linen Big Shirt'							
Select Recommendation							
Original Explain Plan (Annotated)							
Implement							
Select	Type	Findings	Recommendations	Rationale	New Benefit (%)	Explain Plan	Compare Explain Plans
☒	SQL Profile	A potentially better execution plan was found for this statement.	No SQL profile currently exists for this recommendation. Consider accepting the recommended SQL profile.	The SQL profile was not automatically created because auto-creation was disabled. Set task parameter ACCEPT_SQL_PROFILES to TRUE to enable auto-creation.	96.01	SQL	SQL
☐	Index	The execution plan of this statement can be improved by creating one or more indices.	Consider running the Access Advisor to improve the physical schema design or creating the recommended index. SH.PRODUCTS("PROD_NAME") SH.SALES("AMOUNT_SOLD")	Creating the recommended indices significantly improves the execution plan of this statement. However, it might be preferable to run "Access Advisor" using a representative SQL workload as opposed to a single statement. This will allow to get comprehensive index recommendations which takes into account index maintenance overhead and additional space consumption.	62.12	SQL	SQL

This page can include recommendations for SQL profiles and indexes.

Accessing the Database Home Page

The Database Home page is the main database management page in Oracle Enterprise Manager Cloud Control (Cloud Control). After you log in to Cloud Control, you navigate to the Database Home page for the target database you want to manage in Cloud Control.

To access the Database Home page in Cloud Control:

1. Start Cloud Control.

The URL for accessing Cloud Control has the following syntax:

`http://hostname.domain:portnumber/em`

2. In the Welcome page, enter your Cloud Control user name and password, and then click **Login**.
3. From the **Targets** drop-down menu, select **Databases**.

The Databases page appears with a list of the available target databases.

4. Select the database that you want to observe or modify from the Databases page.

If the list of databases is long, use the **Search** functionality.

The home page for the target database appears. The first time that you select an option from some of the menus, such as the Performance menu, the Database Login page appears.

5. In the login page for the target database, log in as a user with the appropriate privileges. For example, to log in as user SYS with the SYSDBA privilege:
 - User Name: Enter SYS.
 - Password: Enter the password for the SYS user.
 - Connect As: From the Role list, select **SYSDBA**.

Moving Data

- Transferring data from one database to another, often when upgrading to a new system or consolidating databases.
- Moving data in Oracle involves several strategies and tools depending on the specific requirements such as the size of the data, the need for minimal downtime, and whether the data is being moved within the same Oracle environment or to a different database system.

Create Directory Objects

- A DIRECTORY object specifies a logical alias name for a physical directory on the database server file system under which the file to be accessed is located. You can access a file in the server file system only if you have the required access privilege on the DIRECTORY object. You can also use Oracle Enterprise Manager Cloud Control to manage the DIRECTORY objects.
- Creating directory objects in Oracle involves creating directory structures within the Oracle Database. This is typically done to facilitate the use of external files by the database, such as for loading data with SQL*Loader, using external tables, or performing data pump operations.
- A directory object is a logical alias for a physical directory path name on the operating system. Directory objects can be created, dropped, and granted access permissions to different users.
- A directory objects act as a layer of abstraction between your database server's file system and your SQL statements.
- A directory objects are schema objects that serve as aliases for directories on the server's file system. These objects are essential for managing and securing file system access within Oracle, providing a controlled way to read from and write to the file system. They are used in various operations, including data loading and unloading, external tables, and accessing binary files stored outside the database.

Key Characteristics of Directory Objects in Oracle

Alias for File System Directories:

A directory object maps to a physical directory on the server's file system. This abstraction allows database users to reference the directory object rather than the file system path directly.

Security and Access Control:

Directory objects enhance security by allowing the DBA to control access through database privileges. Users must have specific privileges (READ, WRITE) on the directory object to access the underlying file system directory.

Common Uses:

- **Data Pump:** For exporting and importing data.
- **External Tables:** To access data stored in external files as if they were database tables.
- **BFILEs:** To reference large binary files stored outside the database.
- **UTL_FILE Package:** To read from and write to files in PL/SQL programs.

Creating and Managing Directory Objects

- Use the **CREATE DIRECTORY** statement to create a directory object. A directory object specifies an alias for a directory on the server file system where external binary file LOBs (BFILEs) and external table data are located. You can use directory names when referring to BFILEs in your PL/SQL code and OCI calls, rather than hard coding the operating system path name, for management flexibility.
- All directories are created in a single namespace and are not owned by an individual schema. You can secure access to the BFILEs stored within the directory structure by granting object privileges on the directories to specific users.
- You must have the **CREATE ANY DIRECTORY** system privilege to create directories.
 - When you create a directory, you are automatically granted the **READ**, **WRITE**, and **EXECUTE** object privileges on the directory, and you can grant these privileges to other users and roles. The DBA can also grant these privileges to other users and roles.
 - **WRITE** privileges on a directory are useful in connection with external tables. They let the grantee determine whether the external table agent can write a log file or a bad file to the directory.

Syntax

```
CREATE [ OR REPLACE ] DIRECTORY directory_name
    [ SHARING = { METADATA | NONE } ]
    AS 'path_name';
```

- **CREATE [OR REPLACE] DIRECTORY:** This part of the statement creates a new directory object. The **OR REPLACE** clause allows the directory object to be recreated if it already exists.
- **directory_name:** The name you assign to the directory object.
- **SHARING = { METADATA | NONE }:** This optional clause specifies whether the directory's metadata can be shared.
- **METADATA:** Allows the directory metadata to be shared, which means it can be accessed by other pluggable databases (PDBs) in a container database (CDB).
- **NONE:** The default setting, indicating that the directory's metadata is not shared.
- **AS 'path_name':** Specifies the absolute path to the directory on the server's file system.

Example:

```
CREATE DIRECTORY my_files AS '/u01/app/oracle/my_files';
```

- This command creates a directory object named **my_files** that points to the **/u01/app/oracle/my_files** path on the server

```
CREATE OR REPLACE DIRECTORY my_files AS '/u01/app/oracle/new_files';
```

- This command creates a directory object named **my_files**. If the directory object already exists, it is replaced with the new path **/u01/app/oracle/new_files**.

```
CREATE DIRECTORY my_shared_files SHARING = METADATA AS
'/u01/app/oracle/shared_files';
```

- This command creates a directory object named **my_shared_files** and allows its metadata to be shared among other PDBs in the CDB.

Granting Privileges

- After creating a directory object, you must grant the necessary privileges to the users who need to access it.

Granting READ and WRITE Privileges

GRANT READ, WRITE ON DIRECTORY my_files TO scott;

- This command grants the user scott read and write access to the my_files directory object.

Data Pump Architecture

- Oracle Data Pump enables high-speed movement of data and metadata from one database to another.
- This technology is the basis for the following Oracle Database data movement utilities:
 - **Data Pump Export (Export)**
 - Export is a utility for unloading data and metadata into a set of operating system files called a dump file set. The dump file set is made up of one or more binary files that contain table data, database object metadata, and control information.
 - **Data Pump Import (Import)**
 - Import is a utility for loading an export dump file set into a database. You can also use Import to load a destination database directly from a source database with no intervening files, which allows export and import operations to run concurrently, minimizing total elapsed time.

Oracle Data Pump is made up of the following distinct parts:

- **The command-line clients *expdp* and *impdp***
These clients make calls to the DBMS_DATAPUMP package to perform Oracle Data Pump operations.
- **The DBMS_DATAPUMP PL/SQL package, also known as the Data Pump API**
This API provides high-speed import and export functionality.
- **The DBMS_METADATA PL/SQL package, also known as the **Metadata API****
All processes that load and unload metadata use this API, which stores object definitions in XML.
- The following figure shows how Oracle Data Pump integrates with SQL*Loader and external tables. As shown, SQL*Loader is integrated with the External Table API and the Data Pump API to load data into external tables. Clients such as Oracle Enterprise

Manager Cloud Control (Cloud Control) and transportable tablespaces can use the Oracle Data Pump infrastructure.

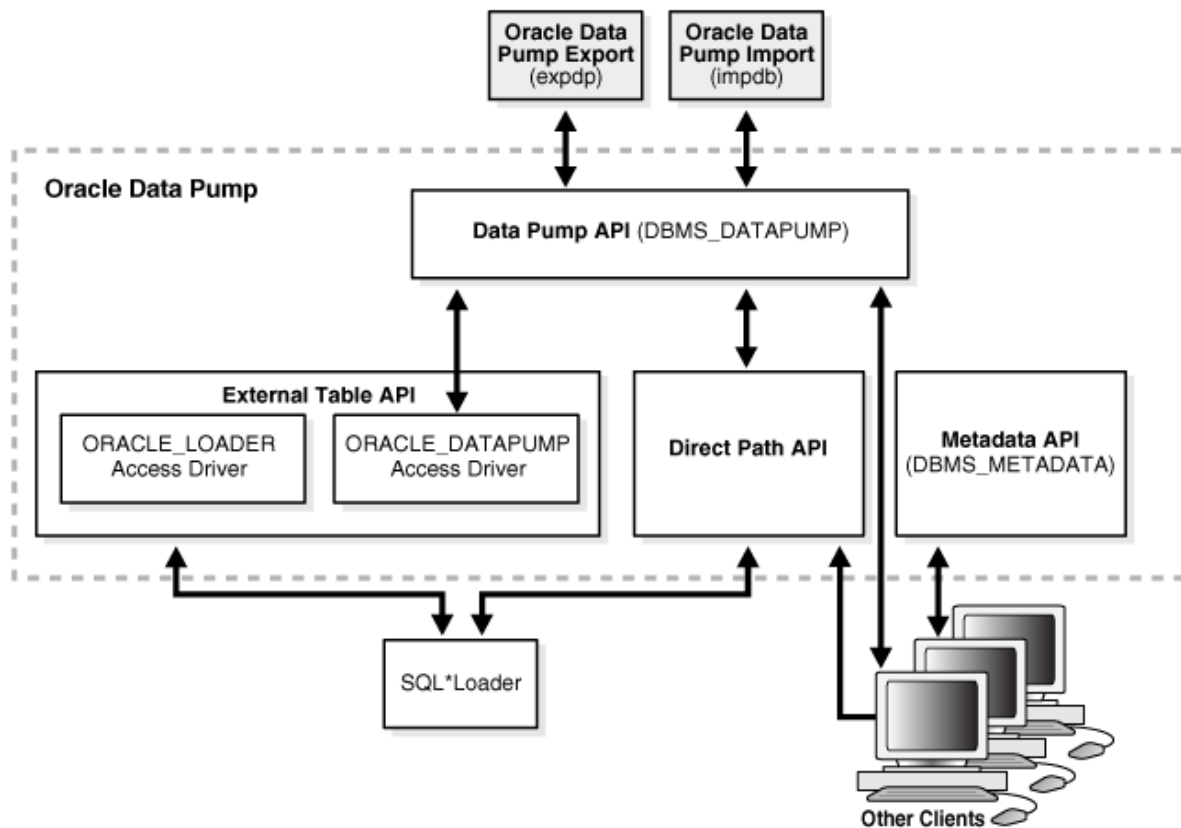


Figure: Oracle Data Pump Architecture

- The Oracle Data Pump clients, **expdp** and **impdp**, start the Oracle Data Pump Export utility and Oracle Data Pump Import utility, respectively.
- The **expdp** and **impdp** clients use the procedures provided in the **DBMS_DATAPUMP PL/SQL** package to execute export and import commands, using the parameters entered at the command line. These parameters enable the exporting and importing of data and metadata for a complete database or for subsets of a database.
- When metadata is moved, Data Pump uses functionality provided by the **DBMS_METADATA PL/SQL** package. The **DBMS_METADATA** package provides a centralized facility for the extraction, manipulation, and re-creation of dictionary metadata.
- The **DBMS_DATAPUMP** and **DBMS_METADATA PL/SQL** packages can be used independently of the Data Pump clients.

Note:

- All Oracle Data Pump Export and Import processing, including the reading and writing of dump files, is done on the system (server) selected by the specified database connect string. This means that for unprivileged users, the database administrator (DBA) must create directory objects for the Data Pump files that are read and written on that server file system. (For security reasons, DBAs must ensure that only approved users are allowed access to directory objects.) For privileged users, a default directory object is available.
- Starting with Oracle Database 18c, you can include the unified audit trail in either full or partial export and import operations using Oracle Data Pump. There is no change to the user interface. When you perform the export or import operations of a database, the unified audit trail is automatically included in the Oracle Data Pump dump files. See Oracle Database PL/SQL Packages and Types Reference for a description of the DBMS_DATAPUMP and the DBMS_METADATA packages. See Oracle Database Security Guide for information about exporting and importing the unified audit trail using Oracle Data Pump.

How Does Oracle Data Pump Move Data?

There are several Oracle Data Pump methods that you can use to move data in and out of databases. You can select the method that best fits your use case.

Note:

The UTL_FILE_DIR desupport in Oracle Database 18c and later releases affects Oracle Data Pump. This desupport can affect any feature from an earlier release using symbolic links, including (but not restricted to) Oracle Data Pump, BFILEs, and External Tables. If you attempt to use an affected feature configured with symbolic links, then you encounter ORA-29283: invalid file operation: path traverses a symlink. Oracle recommends that you instead use directory objects in place of symbolic links.

Data Pump does not load tables with disabled unique indexes. To load data into the table, the indexes must be either dropped or reenabled.

Following are the ways for using Oracle Data Pump to Move Data

- **Using Data File Copying to Move Data**

The fastest method of moving data is to copy the database data files to the target database without interpreting or altering the data.

- **Using Direct Path to Move Data**

After data file copying, direct path is the fastest method of moving data. In this method, the SQL layer of the database is bypassed and rows are moved to and from the dump file with only minimal interpretation.

- **Using External Tables to Move Data**

If you do not select data file copying, and the data cannot be moved using direct path, you can use the external tables mechanism.

- **Using Conventional Path to Move Data**

Where there are conflicting table attributes, Oracle Data Pump uses conventional path to move data.

- **Using Network Link Import to Move Data**

When the Import `NETWORK_LINK` parameter is used to specify a network link for an import operation, the direct path method is used by default. Review supported database link types.

- **Using a Parameter File (Parfile) with Oracle Data Pump**

To help to simplify Oracle Data Pump exports and imports, you can create a **parameter** file, also known as a **parfile**.

Using Data File Copying to Move Data

- The fastest method of moving data is to copy the database data files to the target database without interpreting or altering the data. With this method, Data Pump Export is used to unload only structural information (metadata) into the dump file.
- The `TRANSPORT_TABLESPACES` parameter is used to specify a transportable tablespace export. Only metadata for the specified tablespaces is exported.
- The `TRANSPORTABLE=ALWAYS` parameter is supplied on a table mode export (specified with the `TABLES` parameter) or a full mode export (specified with the `FULL` parameter) or a full mode network import (specified with the `FULL` and `NETWORK_LINK` parameters).
- When an export operation uses data file copying, the corresponding import job always also uses data file copying. During the ensuing import operation, both the data files and the export dump file must be loaded.
- When data is moved by using data file copying, there are some limitations regarding character set compatibility between the source and target databases.

- If the source platform and the target platform are of different endianness, then you must convert the data being transported so that it is in the format of the target platform. You can use the DBMS_FILE_TRANSFER PL/SQL package or the RMAN CONVERT command to convert the data.

Using Direct Path to Move Data

- After data file copying, direct path is the fastest method of moving data. In this method, the SQL layer of the database is bypassed and rows are moved to and from the dump file with only minimal interpretation.
- Data Pump automatically uses the direct path method for loading and unloading data unless the structure of a table does not allow it. For example, if a table contains a column of type BFILE, then direct path cannot be used to load that table and external tables is used instead.
- The following sections describe situations in which direct path cannot be used for loading and unloading.

Situations in Which Direct Path Load Is Not Used

If any of the following conditions exist for a table, then Data Pump uses external tables to load the data for that table, instead of direct path:

- A domain index that is not a CONTEXT type index exists for a LOB column.
- A global index on multipartition tables exists during a single-partition load. This case includes object tables that are partitioned.
- A table is in a cluster.
- There is an active trigger on a preexisting table.
- Fine-grained access control is enabled in insert mode on a preexisting table.
- A table contains BFILE columns or columns of opaque types.
- A referential integrity constraint is present on a preexisting table.
- A table contains VARRAY columns with an embedded opaque type.
- The table has encrypted columns.
- The table into which data is being imported is a preexisting table and at least one of the following conditions exists:
 - There is an active trigger

- The table is partitioned
- Fine-grained access control is in insert mode
- A referential integrity constraint exists
- A unique index exists
- Supplemental logging is enabled, and the table has at least one LOB column.
- The Data Pump command for the specified table used the QUERY, SAMPLE, or REMAP_DATA parameter.
- A table contains a column (including a VARRAY column) with a TIMESTAMP WITH TIME ZONE data type, and the version of the time zone data file is different between the export and import systems.

Situations in Which Direct Path Unload Is Not Used

If any of the following conditions exist for a table, then Data Pump uses external tables rather than direct path to unload the data:

- Fine-grained access control for SELECT is enabled.
- The table is a queue table.
- The table contains one or more columns of type BFILE or opaque, or an object type containing opaque columns.
- The table contains encrypted columns.
- The table contains a column of an evolved type that needs upgrading.
- The Data Pump command for the specified table used the QUERY, SAMPLE, or REMAP_DATA parameter.
- Before the unload operation, the table was altered to contain a column that is NOT NULL, and also has a default value specified.

Using External Tables to Move Data

If you do not select data file copying, and the data cannot be moved using direct path, you can use the external tables mechanism.

The external tables mechanism creates an external table that maps to the dump file data for the database table. The SQL engine is then used to move the data. If possible, use the APPEND hint on import to speed the copying of the data into the database. The representation of data for direct path data and external table data is the same in a dump file. Because they are the same, Oracle Data Pump can use the direct path mechanism at export

time, but use external tables when the data is imported into the target database. Similarly, Oracle Data Pump can use external tables for the export, but use direct path for the import.

In particular, Oracle Data Pump can use external tables in the following situations:

- Loading and unloading very large tables and partitions in situations where it is advantageous to use parallel SQL capabilities
- Loading tables with global or domain indexes defined on them, including partitioned object tables
- Loading tables with active triggers or clustered tables
- Loading and unloading tables with encrypted columns
- Loading tables with fine-grained access control enabled for inserts
- Loading a table not created by the import operation (the table exists before the import starts)

Using Conventional Path to Move Data

- Where there are conflicting table attributes, Oracle Data Pump uses conventional path to move data.
- In situations where there are conflicting table attributes, Oracle Data Pump is not able to load data into a table using either direct path or external tables. In such cases, conventional path is used, which can affect performance.

Using Network Link Import to Move Data

- When the Import NETWORK_LINK parameter is used to specify a network link for an import operation, the direct path method is used by default. Review supported database link types.
- If direct path cannot be used (for example, because one of the columns is a BFILE), then SQL is used to move the data using an INSERT SELECT statement. (Before Oracle Database 12c Release 2 (12.2.0.1), the default was to use the INSERT SELECT statement.) The SELECT clause retrieves the data from the remote database over the network link. The INSERT clause uses SQL to insert the data into the target database. There are no dump files involved.

- When the Export NETWORK_LINK parameter is used to specify a network link for an export operation, the data from the remote database is written to dump files on the target database. (Note that to export from a read-only database, the NETWORK_LINK parameter is required.)
- Because the link can identify a remotely networked database, the terms database link and network link are used interchangeably.

Supported Link Types

The following types of database links are supported for use with Data Pump Export and Import:

- Public fixed user
- Public connected user
- Public shared user (only when used by link owner)
- Private shared user (only when used by link owner)
- Private fixed user (only when used by link owner)

Unsupported Link Types

The following types of database links are not supported for use with Data Pump Export and Import:

- Private connected user
- Current user
- Parallel export or import of metadata for network jobs.

For conventional jobs, if you need parallel metadata import, then use a dumpfile instead of NETWORK_LINK.

Using a Parameter File (Parfile) with Oracle Data Pump

- To help to simplify Oracle Data Pump exports and imports, you can create a parameter file, also known as a parfile.
- Instead of typing in Oracle Data Pump parameters at the command line, when you run an export or import operation, you can prepare a parameter text file (also known as a parfile, after the parameter name) that provides the command-line parameters to the Oracle Data Pump client. You specify that Oracle Data Pump obtains parameters for the command by entering the PARFILE parameter, and then specifying the parameter name:

PARFILE=[directory_path]file_name

- When the Oracle Data Pump Export or Import operation starts, the parameter file is opened and read by the client. The default location of the parameter file is the user's current directory.

For example:

```
expdp hr PARFILE=hr.par
```

- When you create a parameter file, it makes it easier for you to reuse that file for multiple export or import operations, which can simplify these operations, particularly if you perform them regularly. Creating a parameter file also helps you to avoid typographical errors that can occur from typing long Oracle Data Pump commands on the command line, especially if you use parameters whose values require quotation marks that must be placed precisely. On some systems, if you use a parameter file and the parameter value being specified does not have quotation marks as the first character in the string (for example, TABLES=scott."Emp"), then the use of escape characters may not be necessary.
- There is no required file name extension, but Oracle examples use .par as the extension. Oracle recommends that you also use this file extension convention. Using a consistent parameter file extension makes it easier to identify and use these files.

Data Pump Data Dictionary Views

- Oracle Data Pump, a high-speed data movement utility, uses several data dictionary views to manage and track its operations. These views provide valuable insights into the status, progress, and details of Data Pump jobs. Here are some key Data Pump data dictionary views:

☑ **DBA_DATAPUMP_JOBS:** This view lists all Data Pump jobs that have been created in the database. It includes both active and completed jobs.

Important columns:

JOB_NAME: The name of the Data Pump job.

OWNER_NAME: The schema owner of the job.

OPERATION: The type of operation (EXPORT or IMPORT).

JOB_MODE: The mode of the job (FULL, SCHEMA, TABLE, etc.).

STATE: The current state of the job (e.g., EXECUTING, NOT RUNNING, COMPLETED).

- ☑ **DBA_DATAPUMP_SESSIONS:** This view provides information about the sessions associated with Data Pump jobs

Important Columns:

JOB_NAME: The name of the Data Pump job.

SESSION_TYPE: The type of session (e.g., MASTER, WORKER).

SESSION_ID: The ID of the session.

- ☑ **DBA_DATAPUMP_WORKERS:** This view contains information about the worker processes for Data Pump jobs.

Important Columns:

JOB_NAME: The name of the Data Pump job.

PROCESS_NAME: The name of the worker process.

PROCESS_ORDER: The order of the worker process.

- ☑ **DBA_DATAPUMP_JOB_DETAILS:** This view provides detailed information about the Data Pump jobs, such as the job start time, end time, and bytes processed.

Important Columns:

JOB_NAME: The name of the Data Pump job.

STATE: The current state of the job.

DEGREE: The degree of parallelism.

START_TIME: The start time of the job.

END_TIME: The end time of the job.

BYTES_PROCESSED: The number of bytes processed by the job.

- ☑ **DBA_DATAPUMP_MASTER:** This view provides information about the master table used by Data Pump jobs. The master table is a control table that Data Pump uses to track the status and progress of a job.

Important Columns:

JOB_NAME: The name of the Data Pump job.

MASTER_TABLE: The name of the master table.

- ☑ **DBA_DATAPUMP_TRACE:** This view is used to monitor and diagnose issues related to Data Pump jobs. It contains trace information that can help identify problems.

Important Columns:

JOB_NAME: The name of the Data Pump job.

TRACE_FILE: The name of the trace file.

- ☑ **DBA_DATAPUMP_SCHEMAS:** This view lists schemas that are included in or excluded from an export or import Data Pump job.

Important Columns:

JOB_NAME: The name of the Data Pump job.

SCHEMA_NAME: The name of the schema.

INCLUDED: Indicates whether the schema is included or excluded (YES or NO).

- ☑ **DBA_DATAPUMP_JOBS:** As previously mentioned, this view tracks the status and details of completed and running Data Pump jobs.

Important Columns:

JOB_NAME: The name of the Data Pump job.

OWNER_NAME: The schema owner of the job.

OPERATION: The type of operation (EXPORT or IMPORT).

JOB_MODE: The mode of the job (FULL, SCHEMA, TABLE, etc.).

STATE: The current state of the job (e.g., EXECUTING, NOT RUNNING, COMPLETED).

START_TIME: The start time of the job.

END_TIME: The end time of the job.

- ☑ **DBA_DATAPUMP_SETTINGS:** This view shows the current configuration settings for Data Pump jobs. It helps in understanding how the jobs are configured and which parameters are being used.

Important Columns:

SETTING_NAME: The name of the setting.

SETTING_VALUE: The value of the setting.

JOB_NAME: The name of the Data Pump job (if the setting is specific to a job).

OWNER_NAME: The owner of the job or setting.

- ☑ **DBA_DATAPUMP_FILESIZES:** This view provides information about the size of various components within a Data Pump set of files. It is useful for understanding the storage and space utilization of Data Pump operations.

Important Columns:

JOB_NAME: The name of the Data Pump job.

FILE_NAME: The name of the file.

COMPONENT: The specific component within the Data Pump file (e.g., table data, metadata).

FILE_SIZE: The size of the file component.

BYTES_PROCESSED: The number of bytes processed for the component.

Data Pump Interactive Mode

- Oracle Data Pump provides an interactive command-line mode that allows you to monitor and manage Data Pump jobs in real-time. This mode can be particularly useful for pausing, stopping, or modifying jobs while they are running.
- Data Pump's interactive mode offers a way to interact with ongoing export (expdp) or import (impdp) jobs while they're running. This mode provides some control over the job's execution and can be helpful for monitoring progress or making minor adjustments.

Entering Interactive Mode

- There are two primary ways to enter interactive mode for a running Data Pump job:
 - **Interrupting the Client Output:** While the job is running and logs messages to your terminal, press Ctrl+C. This will halt the logging output and place you in the interactive mode prompt, typically denoted by Export> or Import>.
 - **Attaching to an Existing Job:** Use the ATTACH parameter with the expdp or impdp command. Here's the syntax:
 expdp system/password ATTACH=job_name (for export)
 impdp system/password ATTACH=job_name (for import)

Using Interactive Commands

Once in interactive mode, you can use a variety of commands to manage the job. Here are some common commands:

HELP: Displays a list of available commands

Export> HELP

STATUS: Displays the current status of the job, including the percentage completed and the current operation.

Export> STATUS

STOP_JOB: Stops the Data Pump job, which can later be restarted

Export> STOP_JOB

START_JOB: Restarts a stopped Data Pump job

Export> START_JOB

KILL_JOB: Terminates the job immediately and removes the master table and dump files.

Export> KILL_JOB

CONTINUE_CLIENT: Returns to the client session and allows the job to continue running

Export> CONTINUE_CLIENT

EXIT_CLIENT: Exits interactive mode and leaves the job running in the background

Export> EXIT_CLIENT

Example

Start an Export Job:

*expdp system/password@database SCHEMAS=hr DIRECTORY=dump_dir
DUMPFILE=hr_export.dmp LOGFILE=hr_export.log*

Attach to the Job:

expdp ATTACH=SYS_EXPORT_SCHEMA_01

(Assuming SYS_EXPORT_SCHEMA_01 is the job name found from DBA_DATAPUMP_JOBS.)

View Job Status:

Export> STATUS

Pause the Job:

Export> STOP_JOB=IMMEDIATE

Resume the Job:

Export> START_JOB

Exit Interactive Mode but Keep the Job Running:

Export> EXIT_CLIENT

Data Pump API

- You can automate data movement operations by using the Oracle Data Pump PL/SQL API *DBMS_DATAPUMP*.
- The Oracle Data Pump API *DBMS_DATAPUMP* provides a high-speed mechanism that you can use to move all or part of the data and metadata for a site from one Oracle Database to another. The Oracle Data Pump Export and Oracle Data Pump Import utilities are based on the Oracle Data Pump API.
- The Oracle Data Pump API in Oracle is a powerful toolset for high-speed data movement and database migration. It includes a set of PL/SQL packages that allow you to perform export and import operations programmatically.
- The *DBMS_DATAPUMP* package is used to move all, or part of, a database between databases, including both data and metadata.

How Does the Oracle Data Pump Client Interface API Work?

- The main structure used in the client interface is a job handle, which appears to the caller as an integer.
 - Handles are created using the *DBMS_DATAPUMP.OPEN* or *DBMS_DATAPUMP.ATTACH* function. Other sessions can attach to a job to monitor and control its progress. Handles are session specific. The same job can create different handles in different sessions. As a DBA, the benefit of this feature is that you can start up a job before departing from work, and then watch the progress of the job from home.
- Use Oracle Data Pump ***DBMS_DATAPUMP*** job states show to know which stage your data movement job is performing, and what options are available at each stage.

Job State Definitions

Each phase of a job is associated with a state:

- **Undefined** — before a handle is created
- **Defining** — when the handle is first created
- **Executing** — when the *DBMS_DATAPUMP.START_JOB* procedure is running
- **Completing** — when the job has finished its work and the Oracle Data Pump processes are ending
- **Completed** — when the job is completed

- **Stop Pending** — when an orderly job shutdown has been requested
- **Stopping** — when the job is stopping
- **Idling** — the period between the time that a DBMS_DATAPUMP.ATTACH is run to attach to a stopped job, and the time that a DBMS_DATAPUMP.START_JOB is run to restart that job
- **Not Running** — when a Data Pump control job table exists for a job that is not running (has no Oracle Data Pump processes associated with it)

Usage Notes

- Performing DBMS_DATAPUMP.START_JOB on a job in an **Idling** state returns that job to an **Executing** state.
- If all users run DBMS_DATAPUMP.DETACH to detach from a job in the **Defining** state, then the job is totally removed from the database.
- If a job terminates unexpectedly, or if an instance running the job is shut down, and the job was previously in an **Executing** or **Idling** state, then the job is placed in the **Not Running** state. You can then restart the job.
- The Oracle Data Pump control job process is active in the **Defining, Idling, Executing, Stopping, Stop Pending,** and **Completing** states. It is also active briefly in the **Stopped** and **Completed** states. The Data Pump control table for the job exists in all states except the **Undefined** state. Child processes are only active in the **Executing** and **Stop Pending** states, and briefly in the **Defining** state for import jobs.
- Detaching while a job is in the **Executing** state does not halt the job. You can reattach to a running job at any time to resume obtaining status information about the job.
- A Detach can occur explicitly, when the DBMS_DATAPUMP.DETACH procedure is run, or it can occur implicitly when an Oracle Data Pump API session is run down, when the Oracle Data Pump API is unable to communicate with an Oracle Data Pump job, or when the DBMS_DATAPUMP.STOP_JOB procedure is run.
- The **Not Running** state indicates that a Data Pump control job table exists outside the context of a running job. This state occurs if a job is stopped (and likely can restart later), or if a job has terminated in an unusual way. You can also see this state momentarily during job state transitions at the beginning of a job, and at the end of a job before the Oracle Data Pump control job table is dropped. Note that the **Not Running** state is shown only in the views DBA_DATAPUMP_JOBS and USER_DATAPUMP_JOBS. It is never returned by the GET_STATUS procedure.

Valid Job States in Which DBMS_DATAPUMP Procedures Can Be Run

- The following table shows the valid job states in which DBMS_DATAPUMP procedures can be run. The states listed are valid for both export and import jobs, unless otherwise noted.

Procedure Name	Valid States	Description
ADD_FILE	Defining (valid for both export and import jobs) Executing and Idling (valid only for specifying dump files for export jobs)	Specifies a file for the dump file set, the log file, or the SQLFILE output.
ATTACH	Defining, Executing, Idling, Stopped, Completed, Completing, Not Running	Enables a user session to monitor a job, or to restart a stopped job. If the dump file set or Data Pump control job table for the job have been deleted or altered in any way, then the attach fails.
DATA_FILTER	Defining	Restricts data processed by a job.
DETACH	All	Disconnects a user session from a job.
GET_DUMPFILE_INFO	All	Retrieves dump file header information.

GET_STATUS	All, except Completed, Not Running, Stopped, and Undefined	Obtains the status of a job.
LOG_ENTRY	Defining, Executing, Idling, Stop Pending, Completing	Adds an entry to the log file.
METADATA_FILTER	Defining	Restricts metadata processed by a job.
METADATA_REMAP	Defining	Remaps metadata processed by a job.
METADATA_TRANSFORM	Defining	Alters metadata processed by a job.
OPEN	Undefined	Creates a new job.
SET_PARALLEL	Defining, Executing, Idling	Specifies parallelism for a job.
SET_PARAMETER	Defining Note: You can enter the ENCRYPTION_PASSWORD parameter during the Defining and Idling states.	Alters default processing by a job.
START_JOB	Defining, Idling	Begins or resumes execution of a job.
STOP_JOB	Defining, Executing, Idling, Stop Pending	Initiates shutdown of a job.
WAIT_FOR_JOB	All, except Completed, Not Running, Stopped, and Undefined	Waits for a job to end.

What Are the Basic Steps in Using the Oracle Data Pump API?

To use the Oracle Data Pump API, you use the procedures provided in the DBMS_DATAPUMP package.

The following steps list the basic activities involved in using the Data Pump API, including the point in running an Oracle Data Pump job in which you can perform optional steps. The steps are presented in the order in which you would generally perform the activities.

1. To create an Oracle Data Pump job and its infrastructure, run the DBMS_DATAPUMP.OPEN procedure.
2. When you run the procedure, the Oracle Data Pump job is started.
3. Define any parameters for the job.
4. Start the job.
5. (Optional) Monitor the job until it completes.
6. (Optional) Detach from the job, and reattach at a later time.
7. (Optional) Stop the job.
8. (Optional) Restart the job, if desired.

DBMS_DATAPUMP Overview

DBMS_DATAPUMP provides the following support and functionality.

- The source and target databases can have different hardware, operating systems, character sets, time zones, and versions.
- All object types and datatypes existing in Oracle Database 11g and higher are supported.
- Data and metadata can be transferred between databases without using any intermediary files.
- A subset of a database can be moved based upon object type and names of objects.
- Schema names, data file names, tablespace names, and data can be transformed at import time.
- Previously aborted export and import jobs can be restarted without duplicating or omitting any data or metadata from the original job.
- The resources applied to an export or import job can be modified.
- Data in an Oracle proprietary format can be unloaded and loaded.

Major steps in DBMS_DATAPUMP package

- DBMS_DATAPUMP provides the functionality to perform data movement operations such as exporting and importing data and metadata. Here are the main steps to use this package:

- **Create a Job:** This involves starting a new Data Pump job. You specify the type of job (EXPORT or IMPORT) and the job name.

```
DECLARE
    h1 NUMBER;
BEGIN
    h1 := DBMS_DATAPUMP.OPEN(operation => 'EXPORT', job_mode
=> 'SCHEMA', job_name => 'my_job');
END;
/
```

- **Add Objects to the Job:** Define the specific objects to include in the export or import operation.

```
BEGIN
    DBMS_DATAPUMP.ADD_FILE(handle => h1, filename =>
'export.dmp', directory => 'DATA_PUMP_DIR', filetype =>
DBMS_DATAPUMP.KU$_FILE_TYPE_DUMP_FILE);
    DBMS_DATAPUMP.METADATA_FILTER(handle => h1, name =>
'SCHEMA_EXPR', value => 'IN (''HR'')');
END;
/
```

- **Start the Job:** Execute the job to perform the data movement.

```
BEGIN
    DBMS_DATAPUMP.START_JOB(handle => h1);
END;
/
```

- **Monitor the Job:** Optionally, you can monitor the job's progress.

```
DECLARE
    percent_done NUMBER;
    job_state VARCHAR2(30);
BEGIN
    DBMS_DATAPUMP.GET_STATUS(handle => h1, mask =>
DBMS_DATAPUMP.KU$_STATUS_JOB_PROGRESS, timeout =>
DBMS_DATAPUMP.WAIT_FOREVER, job_state => job_state,
percent_done => percent_done);
    DBMS_OUTPUT.PUT_LINE('Job State: ' || job_state);
    DBMS_OUTPUT.PUT_LINE('Percent Done: ' || percent_done);
END;
/
```

- **Detach from the Job:** After starting the job, you can detach from it, allowing it to run in the background.

```
BEGIN

    DBMS_DATAPUMP.DETACH(handle => h1);

END;

/
```

Using Data Pump to Export Data

- To export data using Oracle Data Pump, you can utilize both the **command-line interface (expdp)** and the **PL/SQL API (DBMS_DATAPUMP)**.

Exporting Data Using expdp Command-Line Utility

The **expdp** utility is a command-line tool that simplifies the Data Pump export process. Here's how you can use it to export the HR schema:

1. **Prepare the Directory Object:** Ensure that you have a directory object created in your database that points to a filesystem location where the dump file will be written.

```
CREATE OR REPLACE DIRECTORY DATA_PUMP_DIR AS '/path/to/directory';
```

```
GRANT READ, WRITE ON DIRECTORY DATA_PUMP_DIR TO your_username;
```

2. **Run the expdp Command:**

Open a terminal and execute the following command:

```
expdp your_username/your_password@your_db
SERVICE_NAME=your_service_name schemas=HR directory=DATA_PUMP_DIR
dumpfile=hr_schema_export.dmp logfile=hr_schema_export.log
```

Here's a breakdown of the command:

`your_username/your_password@your_db`: Your database credentials.

`SERVICE_NAME=your_service_name`: The service name of your database.

`schemas=HR`: The schema you want to export.

`directory=DATA_PUMP_DIR`: The directory object where the dump file will be stored.

`dumpfile=hr_schema_export.dmp`: The name of the dump file.

`logfile=hr_schema_export.log`: The name of the log file.

Exporting Data Using the DBMS_DATAPUMP PL/SQL API

The DBMS_DATAPUMP package provides a programmatic way to manage Data Pump jobs within PL/SQL. Below is an example to export the HR schema:

1. **Prepare the Directory Object:** Ensure that you have a directory object created in your database.

```
CREATE OR REPLACE DIRECTORY DATA_PUMP_DIR AS '/path/to/directory';
GRANT READ, WRITE ON DIRECTORY DATA_PUMP_DIR TO your_username;
```

2. Create and Run the Data Pump Export Job:

```
DECLARE
    h1 NUMBER;
BEGIN
    -- Open a new Data Pump job for export
    h1 := DBMS_DATAPUMP.OPEN(operation => 'EXPORT', job_mode => 'SCHEMA',
    job_name => 'EXPORT_HR_JOB');

    -- Add a dump file
    DBMS_DATAPUMP.ADD_FILE(handle => h1, filename => 'hr_schema_export.dmp',
    directory => 'DATA_PUMP_DIR', filetype =>
    DBMS_DATAPUMP.KU$_FILE_TYPE_DUMP_FILE);

    -- Add a log file
    DBMS_DATAPUMP.ADD_FILE(handle => h1, filename => 'hr_schema_export.log',
    directory => 'DATA_PUMP_DIR', filetype =>
    DBMS_DATAPUMP.KU$_FILE_TYPE_LOG_FILE);

    -- Filter to specify the schema to be exported
    DBMS_DATAPUMP.METADATA_FILTER(handle => h1, name => 'SCHEMA_EXPR', value
    => 'IN (''HR'')');

    -- Start the export job
    DBMS_DATAPUMP.START_JOB(handle => h1);

    -- Detach from the job to allow it to run in the background
    DBMS_DATAPUMP.DETACH(handle => h1);
END;
/
```

NOTE: After the export operation completes, review the log file (`hr_schema_export.log`) in the specified directory (`DATA_PUMP_DIR`) to verify the details and confirm that the export was successful.

Using Data Pump to import Data

Importing data using Oracle Data Pump can be done using both the command-line utility (**impdp**) and the PL/SQL API (**DBMS_DATAPUMP**).

Importing Data Using impdp Command-Line Utility

- **Prepare the Directory Object:** Ensure that you have a directory object created in your database that points to a filesystem location where the dump file is located.

```
CREATE OR REPLACE DIRECTORY DATA_PUMP_DIR AS '/path/to/directory';
GRANT READ, WRITE ON DIRECTORY DATA_PUMP_DIR TO your_username;
```

- **Run the impdp Command:**

Open a terminal and execute the following command to import the HR schema from the dump file:

```
impdp your_username/your_password@your_db
SERVICE_NAME=your_service_name schemas=HR directory=DATA_PUMP_DIR
dumpfile=hr_schema_export.dmp logfile=hr_schema_import.log
```

Here's a breakdown of the command:

your_username/your_password@your_db: Your database credentials.

SERVICE_NAME=your_service_name: The service name of your database.

schemas=HR: The schema you want to import.

directory=DATA_PUMP_DIR: The directory object where the dump file is stored.

dumpfile=hr_schema_export.dmp: The name of the dump file.

logfile=hr_schema_import.log: The name of the log file.

Importing Data Using the DBMS_DATAPUMP PL/SQL API

- The DBMS_DATAPUMP package allows you to manage Data Pump jobs programmatically. Below is an example to import the HR schema:

1. **Prepare the Directory Object:** Ensure that you have a directory object created in your database.
2. **Create and Run the Data Pump Import Job:**

```

DECLARE
    h1 NUMBER;
BEGIN
    -- Open a new Data Pump job for import
    h1 := DBMS_DATAPUMP.OPEN(operation => 'IMPORT', job_mode => 'SCHEMA',
job_name => 'IMPORT_HR_JOB');

    -- Specify the dump file
    DBMS_DATAPUMP.ADD_FILE(handle => h1, filename =>
'hr_schema_export.dmp', directory => 'DATA_PUMP_DIR', filetype =>
DBMS_DATAPUMP.KU$_FILE_TYPE_DUMP_FILE);

    -- Specify the log file
    DBMS_DATAPUMP.ADD_FILE(handle => h1, filename =>
'hr_schema_import.log', directory => 'DATA_PUMP_DIR', filetype =>
DBMS_DATAPUMP.KU$_FILE_TYPE_LOG_FILE);

    -- Start the import job
    DBMS_DATAPUMP.START_JOB(handle => h1);

    -- Detach from the job to allow it to run in the background
    DBMS_DATAPUMP.DETACH(handle => h1);
END;
/

```

NOTE: After the import operation completes, review the log file (hr_schema_import.log) in the specified directory (DATA_PUMP_DIR) to verify the details and confirm that the import was successful.

Overview of SQL Loader

- SQL Loader is a high-speed data loading utility provided by Oracle Database. It allows you to efficiently load data from external files into tables in an Oracle database. SQL Loader is particularly useful for bulk loading large volumes of data.

SQL*Loader Features

SQL*Loader loads data from external files into tables of an Oracle database.

It has a powerful data parsing engine that puts little limitation on the format of the data in the data file. You can use SQL*Loader to do the following:

- Load data across a network if your data files are on a different system than the database.
- Load data from multiple data files during the same load session.
- Load data into multiple tables during the same load session.
- Specify the character set of the data.
- Selectively load data (you can load records based on the records' values).
- Manipulate the data before loading it, using SQL functions.
- Generate unique sequential key values in specified columns.
- Use the operating system's file system to access the data files.
- Load data from disk, tape, or named pipe.
- Generate sophisticated error reports, which greatly aid troubleshooting.
- Load arbitrarily complex object-relational data.
- Use secondary data files for loading LOBs and collections.
- Use conventional, direct path, or external table loads.

You can use **SQL*Loader in two ways: with or without a control file**. A control file controls the behavior of SQL*Loader and one or more data files used in the load. Using a control file gives you more control over the load operation, which might be desirable for more complicated load situations. But for simple loads, you can use SQL*Loader without specifying a control file; this is referred to as *SQL*Loader express mode*.

The output of SQL*Loader is an Oracle database (where the data is loaded), a log file, a bad file if there are rejected records, and potentially, a discard file.

The following figure shows an example of the flow of a typical SQL*Loader session that uses a control file.

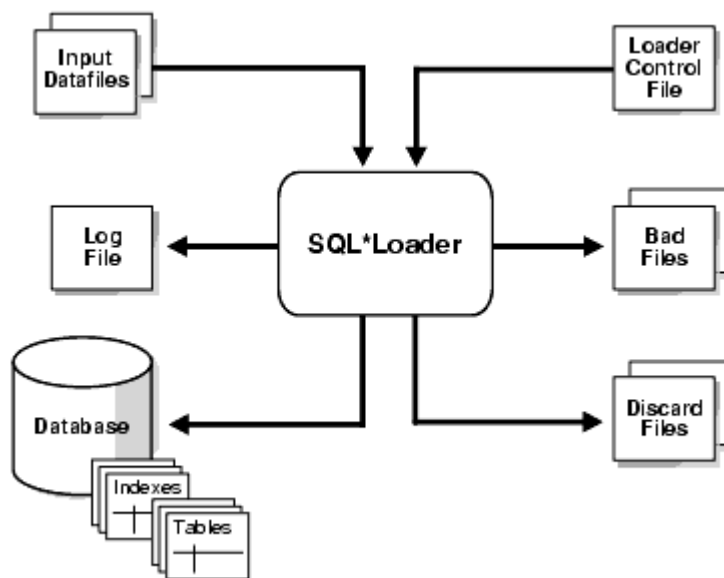


Figure: SQL* Loader Overview

SQL*Loader takes as input a *control file*, which controls the behavior of SQL*Loader, and one or more datafiles. Output of the SQL*Loader is an Oracle database (where the data is loaded), a log file, a bad file, and potentially a discard file.

SQL*Loader Parameters

SQL*Loader is started either when you specify the `sqlldr` command, or when you specify parameters that establish various characteristics of the load operation.

In situations where you always use the same parameters for which the values seldom change, it can be more efficient to specify parameters by using the following methods, rather than on the command line:

- You can group parameters together in a parameter file. You can then specify the name of the parameter file on the command line by using the `PARFILE` parameter.
- You can specify some parameters within the SQL*Loader control file by using the `OPTIONS` clause.

Parameters specified on the command line override any parameter values specified in a parameter file or OPTIONS clause.

SQL*Loader Command-Line Parameters

- To start regular SQL*Loader, use the command-line parameters.

*Note: Regular SQL*Loader and SQL*Loader Express mode share some of the same parameters, but the behavior of these parameters can be different for each utility. The parameter descriptions described here are for regular SQL*Loader. For SQL*Loader Express options, refer to the SQL*Loader Express parameters.*

- To display a help screen that lists all SQL*Loader parameters, enter **sqlldr** at the prompt. and press *Enter*. The output shows each parameter, including default values for parameters, and a brief description of each parameter.

Specifying Parameters on the Command Line

- When you start SQL*Loader, you specify parameters to establish various characteristics of the load operation.
- To see how to specify SQL*Loader parameters, refer to the following examples:

You can separate the parameters by commas. However, it is not required to delimit parameters by commas:

```
> sqlldr CONTROL=ulcase1.ctl LOG=ulcase1.log
```

```
Username: scott
```

```
Password: password
```

- Specifying by position means that you enter a value, but not the parameter name. In the following example, the username scott is provided, and then the name of the control file, ulcase1.ctl. You are prompted for the password:

```
> sqlldr scott ulcase1.ctl
```

```
Password: password
```

- After a parameter name is used, you must supply parameter names for all subsequent specifications. No further positional specification is allowed. For example, in the following command, the CONTROL parameter is used to specify the control file name, but then the log file name is supplied without the LOG parameter, even though the LOG parameter was previously specified. Submitting this command now results in an error, even though the position of ulcase1.log is correct:

```
> sqlldr scott CONTROL=ulcase1.ctl ulcase1.log
```

- For the command to run, you must enter the command with the LOG parameter specifically specified:

```
> sqlldr scott CONTROL=ulcase1.ctl LOG=ulcase1.log
```

SQL*Loader Control File

The control file is a text file written in a language that SQL*Loader understands.

The control file tells SQL*Loader where to find the data, how to parse and interpret the data, where to insert the data, and more.

In general, the control file has three main sections, in the following order:

- Session-wide information
- Table and field-list information
- Input data (optional section)

Some control file syntax considerations to keep in mind are:

- The syntax is free-format (statements can extend over multiple lines).
- The syntax is case-insensitive; however, strings enclosed in single or double quotation marks are taken literally, including case.
- In control file syntax, comments extend from the two hyphens (--) that mark the beginning of the comment to the end of the line. The optional third section of the control file is interpreted as data rather than as control file syntax; consequently, comments in this section are not supported.
- The keywords CONSTANT and ZONE have special meaning to SQL*Loader and are therefore reserved. To avoid potential conflicts, Oracle recommends that you do not use either CONSTANT or ZONE as a name for any tables or columns.

How SQL*Loader Reads Input Data and Data Files

SQL*Loader reads data from one or more data files (or operating system equivalents of files) specified in the control file.

From SQL*Loader's perspective, the data in the data file is organized as records. A particular data file can be in fixed record format, variable record format, or stream record format. The record format can be specified in the control file with the INFILE parameter. If no record format is specified, then the default is stream record format.

Fixed Record Format

- A file is in fixed record format when all records in a data file are the same byte length.
- Although the fixed record format is the least flexible format, using it results in better performance than variable or stream format. Fixed format is also simple to specify. For example:

```
INFILE datafile_name "fix n"
```

This example specifies that SQL*Loader should interpret the particular data file as being in fixed record format where every record is n bytes long.

Variable Record Format and SQL*Loader

- A file is in variable record format when the length of each record in a character field is included at the beginning of each record in the data file.
- This format provides some added flexibility over the fixed record format and a performance advantage over the stream record format. For example, you can specify a data file that is to be interpreted as being in variable record format as follows:

```
INFILE "datafile_name" "var n"
```

In this example, n specifies the number of bytes in the record length field. If n is not specified, then SQL*Loader assumes a length of 5 bytes. Specifying n larger than 40 results in an error.

Stream Record Format and SQL*Loader

- A file is in stream record format when the records are not specified by size; instead SQL*Loader forms records by scanning for the record terminator.
- Stream record format is the most flexible format, but using it can result in a negative effect on performance. The specification of a data file to be interpreted as being in stream record format looks similar to the following:

```
INFILE datafile_name ["str terminator_string"]
```

In the preceding example, str indicates that the file is in stream record format.

The *terminator_string* is specified as either '*char_string*' or X'*hex_string*' where:

- '*char_string*' is a string of characters enclosed in single or double quotation marks

- *X'hex_string'* is a byte string in hexadecimal format

When the *terminator_string* contains special (nonprintable) characters, it should be specified as a *X'hex_string'* byte string. However, you can specify some nonprintable characters as (*'char_string'*) by using a backslash. For example:

- \n indicates a line feed
- \t indicates a horizontal tab
- \f indicates a form feed
- \v indicates a vertical tab
- \r indicates a carriage return

Logical Records and SQL*Loader

- SQL*Loader organizes input data into physical records, according to the specified record format. By default, a physical record is a logical record.
- For added flexibility, SQL*Loader can be instructed to combine several physical records into a logical record.
- SQL*Loader can be instructed to follow one of the following logical record-forming strategies:
 - Combine a fixed number of physical records to form each logical record.
 - Combine physical records into logical records while a certain condition is true.

Data Field Setting and SQL*Loader

- Field setting is a process in which SQL*Loader uses control-file field specifications to determine which parts of logical record data correspond to which control-file fields. It is possible for two or more field specifications to claim the same data. Also, it is possible for a logical record to contain data that is not claimed by any control-file field specification.
- Most control-file field specifications claim a particular part of the logical record. This mapping takes the following forms:

- The byte position of the data field's beginning, end, or both, can be specified. This specification form is not the most flexible, but it provides high field-setting performance.
- The strings delimiting (enclosing, terminating, or both) a particular data field can be specified. A delimited data field is assumed to start where the last data field ended, unless the byte position of the start of the data field is specified.
- You can specify the byte offset, the length of the data field, or both. This way each field starts a specified number of bytes from where the last one ended and continues for a specified length.
- Length-value data types can be used. In this case, the first n number of bytes of the data field contain information about how long the rest of the data field is.

Data Conversion and Data Type Specification

During a conventional path load, *data fields* in the data file are converted into *columns* in the database (direct path loads are conceptually similar, but the implementation is different).

There are two conversion steps:

1. SQL*Loader uses the field specifications in the control file to interpret the format of the data file, parse the input data, and populate the bind arrays that correspond to a SQL INSERT statement using that data. A bind array is an area in memory where SQL*Loader stores data that is to be loaded. When the bind array is full, the data is transmitted to the database. The bind array size is controlled by the SQL*Loader BINDSIZE and READSIZE parameters.
2. The database accepts the data and executes the INSERT statement to store the data in the database.

Oracle Database uses the data type of the column to convert the data into its final, stored form. Keep in mind the distinction between a *field* in a data file and a *column* in the database. Remember also that the field data types defined in a SQL*Loader control file are *not* the same as the column data types.

Record Filtering

- Records read from the input file might not be inserted into the database. Following figure shows the stages at which records may be rejected or discarded.

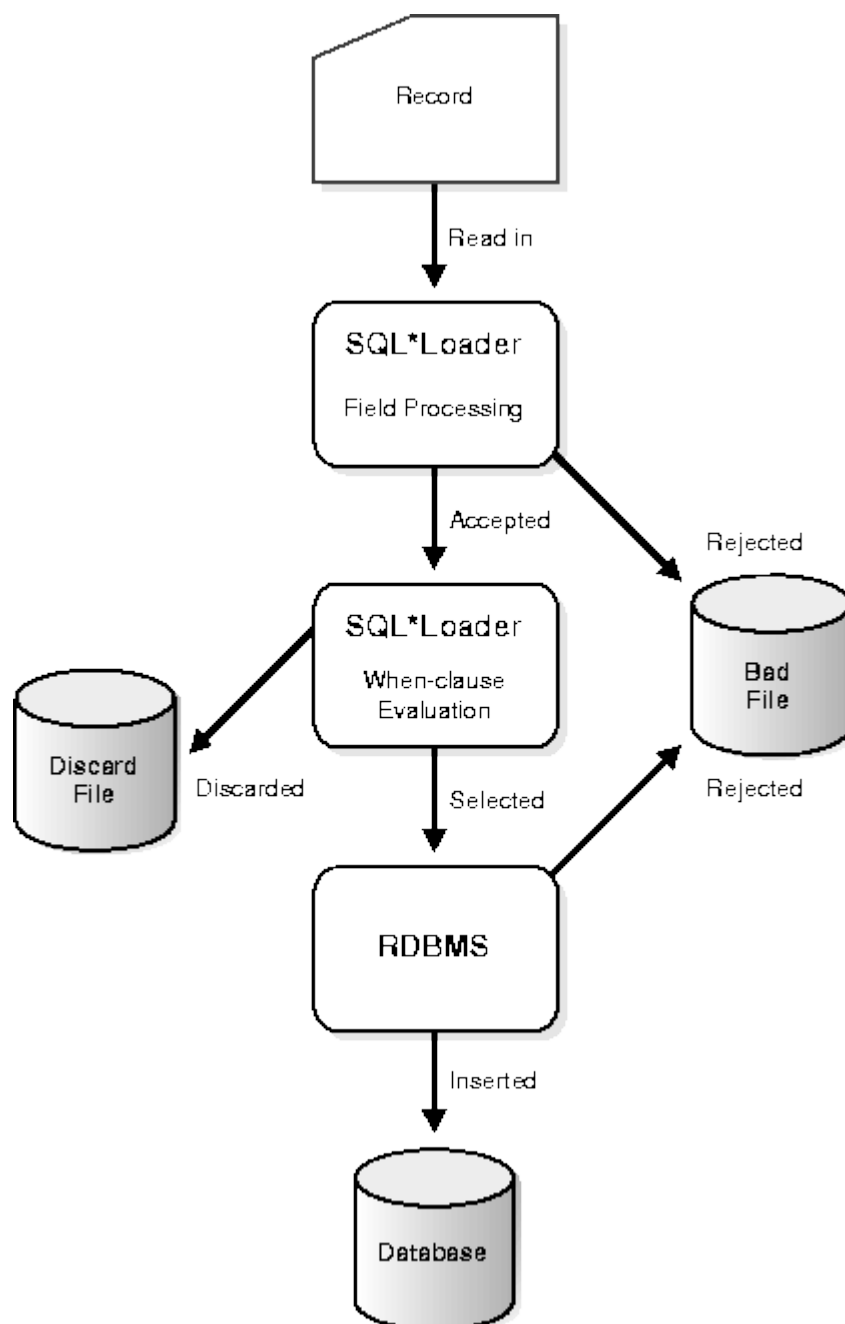


Figure: Record Filtering

- After a record is accepted for processing by SQL*Loader, a row is sent to Oracle for insertion. If Oracle determines that the row is valid, then the row is inserted into the database. If not, the record is rejected, and SQL*Loader puts it in the bad file. The row may be rejected, for example, because a key is not unique, because a required field is null, or because the field contains invalid data for the Oracle datatype.
- The bad file is written in the same format as the datafile. So rejected data can be loaded with the existing control file after necessary corrections are made.
- As SQL*Loader executes, it may create a file called the discard file. This file is created only when it is needed, and only if you have specified that a discard file should be enabled. The discard file contains records that were filtered out of the load because they did not match any record-selection criteria specified in the control file.
- The discard file therefore contains records that were not inserted into any table in the database. You can specify the maximum number of such records that the discard file can accept. Data written to any database table is not written to the discard file.
- The discard file is written in the same format as the datafile. The discard data can be loaded with the existing control file, after any necessary editing or correcting.

SQL*Loader Discarded and Rejected Records

- SQL*Loader can reject or discard some records read from the input file, either because of issues with the files, or because you have selected to filter the records out of the load.
- Rejected records are placed in a bad file, and discarded records are placed in a discard file.

The SQL*Loader Bad File

- The bad file contains records that were rejected, either by SQL*Loader or by Oracle Database.
- If you do not specify a bad file, and there are rejected records, then SQL*Loader automatically creates one. A rejected record has the same name as the data file, with a .bad extension. There can be several causes for rejections.
 - *Records Rejected by SQL*Loader*
Data file records are rejected by SQL*Loader when the input format is invalid.

- *Records Rejected by Oracle Database During a SQL*Loader Operation*

After a data file record is accepted for processing by SQL*Loader, it is sent to the database for insertion into a table as a row.

The SQL*Loader Discard File

- As SQL*Loader runs, it can filter some records out of the load, and create a file called the discard file.
- A discard file is created only when it is needed, and only if you have specified that a discard file should be enabled. The discard file contains records that were filtered out of the load because they did not match any record-selection criteria specified in the control file.
- Because the discard file contains record filtered out of the load, the contents of the discard file are records that were not inserted into any table in the database. You can specify the maximum number of such records that the discard file can accept. Data written to any database table is not written to the discard file.

Log File and Logging Information

- When SQL*Loader begins processing, it creates a log file.
- If SQL*Loader cannot create a log file, then processing terminates. The log file contains a detailed summary of the load, including a description of any errors that occurred during the load.

Control File Keywords

- The SQL*Loader control file is a text file that contains data definition language (DDL) instructions for a SQL*Loader job.
- The SQL*Loader control file is a text file that contains data definition language (DDL) instructions.
- DDL is used to control the following aspects of a SQL*Loader session:
 - Where SQL*Loader will find the data to load
 - How SQL*Loader expects that data to be formatted
 - How SQL*Loader will be configured (memory management, rejecting records, interrupted load handling, and so on) as it loads the data

- How SQL*Loader will manipulate the data being loaded
- In general, the control file has three main sections, in the following order:
 - Session-wide information
 - Table and field-list information
 - Input data (optional section)
- The following is an example of a control file.

```

1  -- This is an example control file
2  LOAD DATA
3  INFILE 'sample.dat'
4  BADFILE 'sample.bad'
5  DISCARDFILE 'sample.dsc'
6  APPEND
7  INTO TABLE emp
8  WHEN (57) = '.'
9  TRAILING NULLCOLS
10 (hiredate SYSDATE,
    deptno POSITION(1:2)  INTEGER EXTERNAL(2)
        NULLIF deptno=BLANKS,
    job      POSITION(7:14)  CHAR  TERMINATED BY WHITESPACE
        NULLIF job=BLANKS  "UPPER(:job)",
    mgr      POSITION(28:31) INTEGER EXTERNAL
        TERMINATED BY WHITESPACE, NULLIF mgr=BLANKS,
    ename    POSITION(34:41) CHAR
        TERMINATED BY WHITESPACE  "UPPER(:ename)",
    empno    POSITION(45)  INTEGER EXTERNAL
        TERMINATED BY WHITESPACE,
    sal      POSITION(51)  CHAR  TERMINATED BY WHITESPACE
        "TO_NUMBER(:sal, '$99,999.99')",
    comm     INTEGER EXTERNAL  ENCLOSED BY '(' AND '%'
        ":comm * 100"
)

```

The numbers that appear to the left in this In this control file example would not appear in a real control file. They are keyed in this sample to the explanatory notes in the following list:

1. This comment prefacing the entries in the control file is an example of how to enter comments in a control file. See [Comments in the Control File](#).
2. The LOAD DATA statement tells SQL*Loader that this is the beginning of a new data load. See [SQL*Loader Syntax Diagrams](#) for syntax information.
3. The INFILE clause specifies the name of a data file containing the data you want to load. See [Specifying Data Files](#).
4. The BADFILE clause specifies the name of a file into which rejected records are placed. See [Specifying the Bad File](#).
5. The DISCARDFILE clause specifies the name of a file into which discarded records are placed. See [Specifying the Discard File](#).

6. The APPEND clause is one of the options that you can use when loading data into a table that is not empty. See [Loading Data into Nonempty Tables](#).
To load data into a table that is empty, use the INSERT clause. See [Loading Data into Empty Tables](#).
7. The INTO TABLE clause enables you to identify tables, fields, and data types. It defines the relationship between records in the data file, and tables in the database. See [Specifying Table Names](#).
8. The WHEN clause specifies one or more field conditions. SQL*Loader decides whether to load the data based on these field conditions. See [Loading Records Based on a Condition](#).
9. The TRAILING NULLCOLS clause tells SQL*Loader to treat any relatively positioned columns that are not present in the record as null columns. See [Handling Short Records with Missing Data](#).
10. The remainder of the control file contains the field list, which provides information about column formats in the table being loaded. See [SQL*Loader Field List Reference](#) for information about that section of the control file.

Specifying Command-Line Parameters in the Control File

- You can specify command-line parameters in the SQL*Loader control file using the OPTIONS clause.
- This can be useful if you often use a control file with the same set of options. The OPTIONS clause precedes the LOAD DATA statement.

OPTIONS Clause

The following command-line parameters can be specified using the OPTIONS clause.

```

BINDSIZE = n
COLUMNARRAYROWS = n
DATE_CACHE = n
DEGREE_OF_PARALLELISM= {degree-num|DEFAULT|AUTO|NONE}
DIRECT = {TRUE | FALSE}
EMPTY_LOBS_ARE_NULL = {TRUE | FALSE}
ERRORS = n
EXTERNAL_TABLE = {NOT_USED | GENERATE_ONLY | EXECUTE}
FILE = tablespace file
LOAD = n
MULTITHREADING = {TRUE | FALSE}
PARALLEL = {TRUE | FALSE}
READSIZE = n
RESUMABLE = {TRUE | FALSE}
RESUMABLE_NAME = 'text string'
RESUMABLE_TIMEOUT = n
ROWS = n
SDF_PREFIX = string
SILENT = {HEADER | FEEDBACK | ERRORS | DISCARDS | PARTITIONS | ALL}
SKIP = n
SKIP_INDEX_MAINTENANCE = {TRUE | FALSE}
SKIP_UNUSABLE_INDEXES = {TRUE | FALSE}
STREAMSIZE = n
TRIM= {LRTRIM|NOTRIM|LTRIM|RTRIM|LDRTRIM}

```

The following is an example use of the OPTIONS clause that you could use in a SQL*Loader control file:

```
OPTIONS (BINDSIZE=100000, SILENT=(ERRORS, FEEDBACK) )
```

Note:Parameter values specified on the command line override parameter values specified in the control file OPTIONS clause.

Specifying the Number of Default Expressions to Be Evaluated At One Time

- Use the SQL*Loader DEFAULT EXPRESSION CACHE n clause to specify how many default expressions are evaluated at a time by the direct path load. The default value is 100.
- Using the DEFAULT EXPRESSION CACHE clause can significantly improve performance when default column expressions that include sequences are evaluated.

- At the end of the load there may be sequence numbers left in the cache that never get used. This can happen when the number of rows to load is not a multiple of n. If you require no loss of sequence numbers, then specify a value of 1 for this clause.

Data Files

In SQL Loader, the **data file** is the file that contains the data to be loaded into an Oracle database. This file can be in various formats such as delimited, fixed-width, or even complex formats. The control file, which directs the SQL Loader operation, describes the format and structure of the data file.

Some data file formats

➤ *Delimited Data File*

A common format is the delimited data file where fields are separated by a specific character, such as a comma (CSV format).

➤ *Fixed-Width Data File*

In a fixed-width data file, each field has a fixed width and data is padded with spaces if necessary.

Key Elements of a Data File:

- **Structure:** The data file should be organized in a way that aligns with the target table structure in your database. Each line (record) in the data file typically represents a row in the table, and the columns within each record correspond to the table's columns.
- **Delimiters:** For flat files, delimiters like commas, tabs, or fixed positions separate data values within each record. SQL Loader uses these delimiters to parse the data and assign values to the appropriate columns.
- **Data Types:** While the data file might have implicit data types based on its format (e.g., numbers, strings), SQL Loader relies on the control file to define the expected data types for each column. This ensures proper conversion and loading of data into the database table.

Using a Data File in SQL*Loader

- To use a data file in SQL*Loader, you need to create a control file that describes the data file format and the target database table.
- To load data files with SQL*Loader, you can specify data files in the control file using the INFILE keyword.

- To specify a data file that contains the data that you want to load, use the INFILE keyword, followed by the file name, and the optional file processing options string.
- To load data from multiple data files in one SQL*Loader run, use an INFILE clause for each data file.

```
INFILE mydat1.dat BADFILE mydat1.bad DISCARDFILE mydat1.dis
INFILE mydat2.dat
INFILE mydat3.dat DISCARDFILE mydat3.dis
INFILE mydat4.dat DISCARDMAX 10 0
```

- For mydat1.dat, both a bad file and discard file are explicitly specified. Therefore both files are created, as needed.
 - For mydat2.dat, neither a bad file nor a discard file is specified. Therefore, only the bad file is created, as needed. If created, the bad file has the default file name and extension mydat2.bad. The discard file is *not* created, even if rows are discarded.
 - For mydat3.dat, the default bad file is created, if needed. A discard file with the specified name (mydat3.dis) is created, as needed.
 - For mydat4.dat, the default bad file is created, if needed. Because the DISCARDMAX option is used, SQL*Loader assumes that a discard file is required and creates it with the default name mydat4.dsc.
- To direct SQL*Loader to access the data files as comma-separated-values format files, use the CSV clause.
 - This assumes that the file is a stream record format file with the normal carriage return string (for example, \n on UNIX or Linux operating systems and either \n or \r\n on Windows operating systems). Record terminators can be included (embedded) in data values. The syntax for the CSV clause is as follows:


```
FIELDS CSV [WITH EMBEDDED|WITHOUT EMBEDDED] [FIELDS
TERMINATED BY ','] [OPTIONALLY ENCLOSED BY '"']
```

The following are key points regarding the **FIELDS CSV clause**:

- The SQL*Loader default is to not use the FIELDS CSV clause.
- The WITH EMBEDDED and WITHOUT EMBEDDED options specify whether record terminators are included (embedded) within any fields in the data.
- If WITH EMBEDDED is used, then embedded record terminators must be enclosed, and intra-datafile parallelism is disabled for external table loads.
- The TERMINATED BY ';' and OPTIONALLY ENCLOSED BY '"' options are the defaults and do not have to be specified. You can override them with different termination and enclosure characters.
- When the CSV clause is used, only delimitable data types are allowed as control file fields. Delimitable data types include CHAR, datetime, interval, and numeric EXTERNAL.
- The TERMINATED BY and ENCLOSED BY clauses cannot be used at the field level when the CSV clause is specified.
- When the CSV clause is specified, normal SQL*Loader blank trimming is done by default. You can specify PRESERVE BLANKS to avoid trimming of spaces. Or, you can use the SQL functions LTRIM and RTRIM in the field specification to remove left and/or right spaces.
- When the CSV clause is specified, the INFILE * clause is not allowed. This means that there cannot be any data included in the SQL*Loader control file.

The following sample SQL*Loader control file uses the FIELDS CSV clause with the default delimiters:

```
LOAD DATA
  INFILE "mydata.dat"
  TRUNCATE
  INTO TABLE mytable
  FIELDS CSV WITH EMBEDDED
  TRAILING NULLCOLS
```

```
(
    c0 char,
    c1 char,
    c2 char,
)
```

SQL Loader Data Paths

- In Oracle SQL*Loader, data paths determine how data is loaded into the database. There are two main data paths: conventional path and direct path

Conventional Path Loads, Direct Path Loads, and External Table Loads

SQL*Loader provides several methods to load data.

- **Conventional Path Loads**

During conventional path loads, the input records are parsed according to the field specifications, and each data field is copied to its corresponding bind array (an area in memory where SQL*Loader stores data to be loaded).

- **Direct Path Loads**

A direct path load parses the input records according to the field specifications, converts the input field data to the column data type, and builds a column array.

- **Parallel Direct Path**

A parallel direct path load allows multiple direct path load sessions to concurrently load the same data segments (allows intrasegment parallelism).

- **External Table Loads**

External tables are defined as tables that do not reside in the database, and can be in any format for which an access driver is provided.

Conventional Path Loads

- During conventional path loads, the input records are parsed according to the field specifications, and each data field is copied to its corresponding bind array (an area in memory where SQL*Loader stores data to be loaded).
- When the bind array is full (or no more data is left to read), an array insert operation is performed.

- SQL*Loader stores LOB fields after a bind array insert is done. Thus, if there are any errors in processing the LOB field (for example, the LOBFILE could not be found), then the LOB field is left empty. Note also that because LOB data is loaded after the array insert has been performed, BEFORE and AFTER row triggers may not work as expected for LOB columns. This is because the triggers fire before SQL*Loader has a chance to load the LOB contents into the column. For instance, suppose you are loading a LOB column, C1, with data and you want a BEFORE row trigger to examine the contents of this LOB column and derive a value to be loaded for some other column, C2, based on its examination. This is not possible because the LOB contents will not have been loaded at the time the trigger fires.

When to Use a Conventional Path Load

- To determine when you should use conventional path load instead of direct path load, review the options for your use case scenario.
- If load speed is most important to you, then you should use direct path load because it is faster than conventional path load. However, certain restrictions on direct path loads can require you to use a conventional path load. You should use a conventional path load in the following situations:
 - When accessing an indexed table concurrently with the load, or when applying inserts or updates to a nonindexed table concurrently with the load
 Note: To use a direct path load (except for parallel loads), SQL*Loader must have exclusive write access to the table and exclusive read/write access to any indexes.
 - When loading data into a clustered table

Reason: A direct path load does not support loading of clustered tables.

- When loading a relatively small number of rows into a large indexed table

Reason: During a direct path load, the existing index is copied when it is merged with the new index keys. If the existing index is very large and the number of new keys is very small, then the index copy time can offset the time saved by a direct path load.

- When loading a relatively small number of rows into a large table with referential and column-check integrity constraints

Reason: Because these constraints cannot be applied to rows loaded on the direct path, they are disabled for the duration of the load. Then they are applied to the whole table when the load completes. The costs could outweigh the savings for a very large table and a small number of new rows.

- When loading records, and you want to ensure that a record is rejected under any of the following circumstances:
 - If the record causes an Oracle error upon insertion
 - If the record is formatted incorrectly, so that SQL*Loader cannot find field boundaries
 - If the record violates a constraint, or a record tries to make a unique index non-unique

*In conventional path loads, SQL*Loader reads data from the input files and uses SQL INSERT statements to load data into the database tables. During this process, input records are parsed according to field specifications, and each data field is copied to a corresponding bind array in memory. The bind array is then used to insert data into the table.*

Characteristics:

- *Uses SQL INSERT statements.*
- *Generates redo and undo log entries, ensuring data recoverability.*
- *Enforces all database constraints (e.g., primary keys, foreign keys, unique constraints).*
- *Triggers are fired during the load process.*
- *Suitable for small to moderate volumes of data.*

Example:

Control File (control.ctl):

```
LOAD DATA

INFILE 'data.txt'

INTO TABLE employees
```

```

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
(
    emp_id    INTEGER EXTERNAL,
    emp_name  CHAR,
    emp_salary DECIMAL EXTERNAL
)

```

Data File (data.txt):

```

1001,John Doe,50000
1002,Jane Smith,60000
1003,Jim Brown,55000

```

Command:

```

sqlldr userid=your_username/your_password@your_db
control=control.ctl log=load.log

```

Direct Path Loads

A direct path load parses the input records according to the field specifications, converts the input field data to the column data type, and builds a column array.

The column array is passed to a block formatter, which creates data blocks in Oracle database block format. The newly formatted database blocks are written directly to the database, bypassing much of the data processing that normally takes place. Direct path load is much faster than conventional path load, but entails several restrictions.

When you use the direct path load feature of SQL*Loader, then instead of filling a bind array buffer and passing it to the Oracle database with a SQL INSERT statement, a direct path load uses the direct path API to pass the data to be loaded to the load engine in the server. The load engine builds a column array structure from the data passed to it.

The direct path load engine uses the column array structure to format Oracle Database data blocks, and to build index keys. The newly formatted database blocks are written directly to the database (multiple blocks per I/O request using asynchronous writes if the host platform supports asynchronous I/O).

Internally, multiple buffers are used for the formatted blocks. While one buffer is being filled, one or more buffers are being written if asynchronous I/O is available on the host platform. Overlapping computation with I/O increases load performance.

Advantages of a Direct Path Load

Direct path loads typically are faster than using conventional path loads.

A direct path load is faster than the conventional path for the following reasons:

- Partial blocks are not used, so no reads are needed to find them, and fewer writes are performed.
- SQL*Loader does not need to run any SQL INSERT statements; therefore, the processing load on Oracle Database is reduced.
- A direct path load calls on Oracle Database to lock tables and indexes at the start of the load, and release those locks when the load is finished. A conventional path load issues an Oracle Database call once for each array of rows to process a SQL INSERT statement.
- A direct path load uses multiblock asynchronous I/O for writes to the database files.
- During a direct path load, processes perform their own write I/O, instead of using the Oracle Database buffer cache. This process method minimizes contention with other Oracle Database users.
- The sorted indexes option available during direct path loads enables you to presort data using high-performance sort routines that are native to your system or installation.
- When a table that you specify to load is empty, the presorting option eliminates the sort and merge phases of index-building. The index is filled in as data arrives.
- Protection against instance failure does not require redo log file entries during direct path loads. Therefore, no time is required to log the load when:

- Oracle Database has the SQL NOARCHIVELOG parameter enabled
- The SQL*Loader UNRECOVERABLE clause is enabled
- The object being loaded has the SQL NOLOGGING parameter set

Restrictions on Using Direct Path Loads

To use the direct path load method, your tables and segments must meet certain requirements. Some features are not available with Direct Path Loads.

The following conditions must be satisfied for you to use the direct path load method:

- Tables that you want to load cannot be clustered.
- Tables that you want to load cannot have Oracle Virtual Private Database (VPD) policies active on INSERT.
- Segments that you want to load cannot have any active transactions pending.

To check for active transactions, use the Oracle Enterprise Manager command MONITOR TABLE to find the object ID for the tables that you want to load. Then use the command MONITOR LOCK to see if there are any locks on the tables.

- For Oracle Database releases earlier than Oracle9i, you can perform a SQL*Loader direct path load only when the client and server are the same release. This restriction also means that you cannot perform a direct path load of Oracle9i data into an earlier Oracle Database release. For example, you cannot use direct path load to load data from Oracle Database 9i Release 1 (9.0.1) into an Oracle 8i (8.1.7) Oracle Database. Beginning with Oracle Database 9i, you can perform a SQL*Loader direct path load when the client and server are different releases. However, both releases must be at least Oracle Database 9i Release 1 (9.0.1), and the client release must be the same as or lower than the server release.

The following features are not available with direct path load:

- Loading BFILE columns
- Use of CREATE SEQUENCE during the load. This is because in direct path loads there is no SQL being generated to fetch the next value, because direct path does not generate INSERT statements.

When to Use a Direct Path Load

If you are not restricted by views, field defaults, or integrity constraints, then then you should use a direct path load in the following circumstances:

- You have a large amount of data to load quickly. A direct path load can quickly load and index large amounts of data. It can also load data into either an empty or nonempty table.
- You want to load data in parallel for maximum performance.

Direct path loads bypass the SQL engine and write data directly into the database files. This method parses input records, converts input field data to column data types, and builds a column array, which is then written directly to the data files.

Characteristics:

- Bypasses the SQL processing layer, writing directly to data files.
- Reduces logging (redo and undo), but minimal logging may occur for indexes and constraints.
- Only NOT NULL constraints are enforced; other constraints and triggers are bypassed.
- Suitable for large volumes of data and faster loading compared to the conventional path.

Example:Control File (control.ctl):

```

LOAD DATA
  INFILE 'data.txt'
  INTO TABLE employees
  FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
  (
    emp_id      INTEGER EXTERNAL,
    emp_name    CHAR,
    emp_salary  DECIMAL EXTERNAL
  )

```

Data File (data.txt):

```

1001,John Doe,50000
1002,Jane Smith,60000
1003,Jim Brown,55000

```

Command:

```

sqlldr userid=your_username/your_password@your_db
control=control.ctl log=load.log direct=true

```

Parallel Direct Path

- A parallel direct path load allows multiple direct path load sessions to concurrently load the same data segments (allows intrasegment parallelism).
- Parallel direct path is more restrictive than direct path.

*Parallel direct path loads allow multiple SQL*Loader sessions to concurrently load data into the same table segments. This is achieved through intrasegment parallelism, enhancing performance and reducing load time for very large datasets.*

Characteristics:

- *Multiple direct path load sessions work simultaneously.*
- *Efficient for large-scale data loading operations.*
- *Requires careful planning of segment allocation and table partitioning to avoid contention.*

Example

```
sqlldr userid=your_username/your_password@your_db control=control.ctl log=load.log
direct=true parallel=true
```

External Table Loads

- External tables are defined as tables that do not reside in the database, and can be in any format for which an access driver is provided.
- Oracle Database provides two access drivers: ORACLE_LOADER, and ORACLE_DATAPUMP. By providing the database with metadata describing an external table, the database is able to expose the data in the external table as if it were data residing in a regular database table.
- An external table load creates an external table for data that is contained in an external data file. The load runs INSERT statements to insert the data from the data file into the target table.
- The advantages of using external table loads over conventional path and direct path loads are as follows:
 - If a data file is big enough, then an external table load attempts to load that file in parallel.
 - An external table load allows modification of the data being loaded by using SQL functions and PL/SQL functions as part of the INSERT statement that is used to create the external table.

External tables allow Oracle to access data stored outside the database as if it were inside a table. This method relies on Oracle's external table functionality and can handle various file formats using access drivers.

Characteristics:

- *Data remains external; Oracle reads it as if from a table.*
- *Supports diverse formats and access drivers.*
- *Useful for data warehousing and ETL processes.*
- *Provides the flexibility to perform SQL operations directly on external data.*

Example:

DDL to Create External Table:

```
CREATE TABLE employees_ext (

    emp_id    NUMBER,

    emp_name  VARCHAR2(100),

    emp_salary NUMBER

)

ORGANIZATION EXTERNAL (

    TYPE ORACLE_LOADER

    DEFAULT DIRECTORY data_pump_dir

    ACCESS PARAMETERS (

        RECORDS DELIMITED BY NEWLINE

        FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'

        MISSING FIELD VALUES ARE NULL

    (

        emp_id    INTEGER EXTERNAL,

        emp_name  CHAR,

        emp_salary DECIMAL EXTERNAL

    )

    )

    LOCATION ('data.txt')

)

REJECT LIMIT UNLIMITED;
```